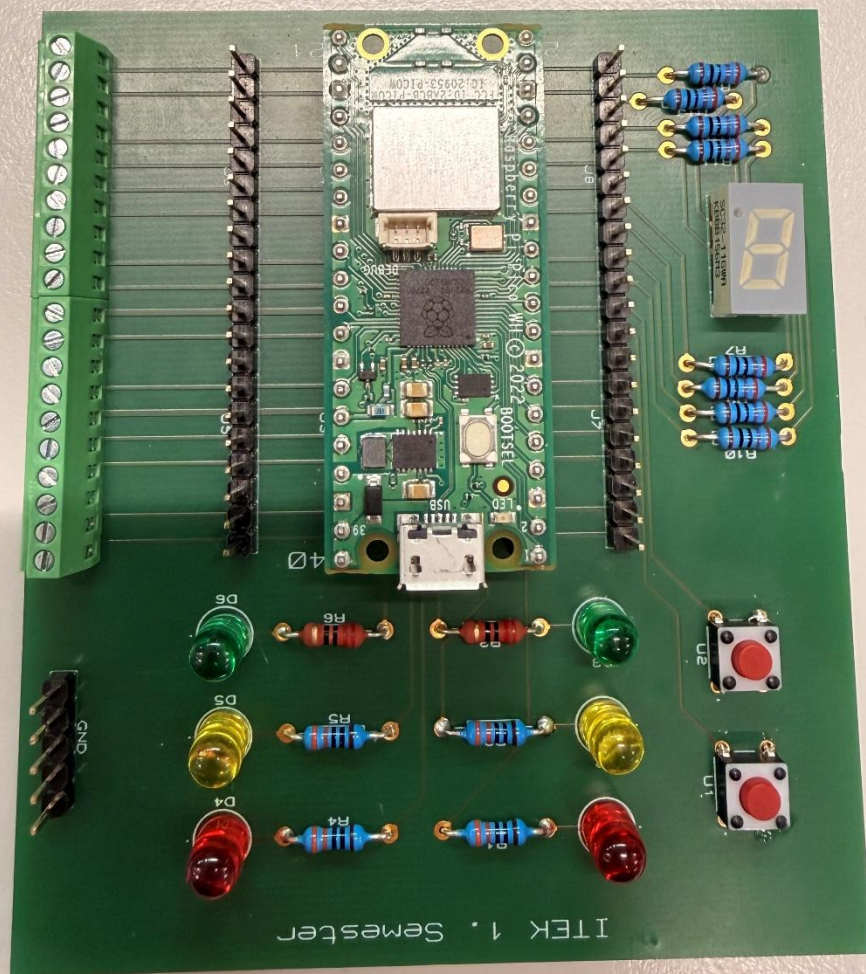


Embedded bibel for ITEK



Indhold

Figur oversigt.....	8
Historisk tidslinje.....	10
Chip-fabrikanter (producerer/støber chippene)	10
Chip-leverandører 2021 (efter salg)	10
Praktisk tjekliste	11
Multimeter og voltmeter	12
Eksempel: simpelt konsol-voltmeter	12
Hardware: Pico 2 W og printet	13
Raspberry Pi Pico 2 W — nøgletal.....	13
Pin-layout for LED'erne	15
Pin-layout for Knapperne.....	16
Sådan sættes det i MicroPython	16
PULL_UP: knappen forbundet til stel	16
PULL_UP: knappen forbundet til 3,3 V	17
Pin-layout for 7-segment display	18
Pin-layout for ADC	19
Oversigt over Big Beautiful Board repository exam kode.....	20
A/D, ADC OG Batteri-spændingsindikator programmer	20
battery_voltage_indicator.py.....	20
battery_voltage_getint_style.py.....	20
Tidligere eksamensbesvarelser	21
20_points.py	21
incomplete_30_points_worth_15points.py	21
fixed_30_points_pwm.py.....	22
Trafiklys- og lyskryds-programmer	22
traffic_light_single.py	22
traffic_light_getint_style.py.....	23
traffic_light_typed_timing.py	23
lyskryds_intersection.py	23

Lyskryds_getint_style.py.....	24
Statemachine	24
trafiklys_enkelt.py.....	24
lyskryds_kryds_statemachine.py	25
lyskryds_med_knapper_statemachine.py.....	25
statemachine_trafiklys_enkelt.py	26
statemachine_lyskryds_kryds.py	26
statemachine_elevator.py	26
statemachine_fake_termostat.py	27
statemachine_hysteresis_termostat.py	27
tilfældig_lyskryds_statemachine.py	28
random_statemachine_1_.py	28
random_statemachine_random_getint_style.py.....	29
pwm_fysisk_knap_lysstyrke.py	29
adc_spændingsmåler.py	29
adc_batteri_spændingsdeler_statemachine.py.....	30
analogt_voltmeter_led_bar.py	30
7segment_nedtælling_statemachine.py.....	31
gy53_afstandsmåling_statemachine.py	31
PWM-programmer	31
pwm_single_led.py.....	32
pwm_three_leds.py	32
pwm_two_leds_independent.py	32
pwm_buttons.py.....	33
pwm_potentiometer_brightness.py.....	33
pwm_traffic_light_sequence.py	34
pwm_morse_sos.py.....	34
pwm_simon_says.py.....	34
pwm_two_player_race.py.....	35
pwm_wave_breathing.py.....	35

pwm_hold_to_ramp.py.....	36
pwm_knight_rider.py.....	36
pwm_dimmer_presets.py.....	36
pwm_reaction_game.py.....	37
pwm_buzzer_tone.py.....	37
pwm_crossfade_two_leds.py.....	38
7-segment display-programmer.....	38
seven_segment_digits.py.....	38
seven_segment_hex.py.....	38
type_a_digit.py.....	39
blink_digit.py.....	39
scroll_number.py.....	40
countdown.py.....	40
cycle_demo.py.....	40
stopwatch.py.....	41
guess_the_number.py.....	41
Pulse Width Modulation (PWM) teori.....	42
Duty cycle (arbejdscyklus).....	42
PWM: Spændingsregulering.....	42
Duty cycle visualiseret.....	43
Raspberry Pi Pico — PWM-kanaler.....	44
Eksempel: åndende LED (fade ind og ud).....	44
Eksempel: lysstyrke styret af et potentiometer (ADC + PWM).....	45
Mål duty cycle med et oscilloskop.....	45
ADC, A/D og spændingsdeler teori.....	47
Sådan tænker du en spændingsdeler.....	48
Statemachines teori.....	49
Eksempel: hysteresese ved en termostat.....	49
Eksempel på en statemachine:.....	50
Farvet konsoloutput (ANSI escape-koder).....	51

Random statemachine	51
Eksempel: random_statemachine_random_getint_style.py	52
7-segment display teori	54
Eksempel 1: cifferopslag som separat modul.....	54
Eksempel 2: tænd pinene for cifret 0	54
GY-53 ToF-afstandssensor teori	55
Netværkssværktøjer	56
Bageopskrift med PWM-eksempel.....	57
1. Importér det du skal bruge	57
2. Opret dine pins.....	57
3. Sæt PWM-frekvensen	57
4. Skriv selve løkken	57
5. Det færdige program	58
Variation 1: brug en knap i stedet for tid	58
Variation 2: flere "faser" (trafiklys, elevator, hysteres).....	58
Genbrugelige skabeloner	59
1. Input med validering	59
2. Statemachine	59
Eksempel 1: Forenklet trafiklys, som statemachine	60
3. ADC og spænding.....	61
Eksamen, points og svar	62
Ohms lov teori, formler og regneeksempler	66
Ohms lov formler	66
Modstand i Serieforbindelse.....	66
Modstand i Parallelforbindelse	66
Modstand i Serie- og Parallelforbindelse	67
Ohms lovs trekanter	67
Oversigt over ohms love	68
Præfiks	68
Enheder til Ohms lov	69

Hvad siger Ohms lov?.....	69
Ohms lov Formel	69
Sådan taster du det på lommeregneren	70
Eksempler på ohms lov	70
Eksempel 1: find spændingen (U)	70
Eksempel 2: find strømmen (I)	71
Eksempel 3: find modstanden (R)	71
Eksempel 4: Find strømmen (I) 2	72
Eksempler på modstand i serieforbindelse, parallelforbindelse og serie- og parallelforbindelse	72
Serieforbindelse	72
Eksempel: 2 modstande i serie	73
Modstand i parallelforbindelse	74
Eksempel — to modstande i parallel	74
Modstand i serie- og parallelforbindelse	75
Figur a: serieforbindelse inde i en parallelforbindelse	75
Eksempel: figur a med tal	76
Figur b: parallelforbindelse inde i en serieforbindelse	76
Sådan taster du Ohms lov ind på lommeregneren	77
Udregningseksempler fra undervisning	78
2 modstande i serie, hvor 1 er sat i parallel	78
Kredsløbsbeskrivelse	78
Trin 1: Total modstand (R_{Total})	79
Trin 2: Total strømstyrke (IT)	79
Trin 3: Spændingsfald over R_1 (V_1)	79
Trin 4: Spænding over parallelforbindelsen (V_2 og V_3)	79
Trin 5: Strømstyrke i de enkelte grene (I_2 og I_3)	79
2 serieforbindelser sat i parallel med hinanden	80
Den samlede modstand (R_{total})	80
Serieforbindelserne først	80
Parallelforbindelsen til sidst	81

Strømmen i grenene (I)	81
Eksempel med fiktive tal	81
Sådan måler du modstand med multimeter uden strøm	82
1 Udstyr og forberedelse	82
2 Sådan stiller du multimeteret op	82
Sådan aflæser og udfører du selve målingen	82
3 Måling af serieforbindelse	83
Formel for serieforbindelse	85
Udregningseksempel	85
4 Måling af parallelforbindelse	85
Formel for parallelforbindelse	86
Udregningseksempel	87
5 Måling af kombineret serie- og parallelforbindelse	87
Metode: regn i trin	89
Udregningseksempel	89
6 Tjekliste før du måler	89
7 Sammenligning: teoretisk vs. målt værdi	89
Sådan måler du modstand med multimeter med strøm	90
Sådan stiller du multimeteret op til strømmåling	90
Sådan kobler du multimeteret ind i kredsløbet	91
Formel: Ohms lov (strøm)	91
Udregningseksempel	91
Vigtige sikkerhedspunkter	92
Tjekliste før du måler	92
Litteraturliste	93
Undervisningsmateriale	94

Figur oversigt

Figure 1, diagram over BBB.....	13
Figure 2, 7 segment display with GPIO	18
Figure 3, PWM led fra on til off.....	42
Figure 4, duty cycle graf	43
Figure 5, diagram for analog til digital konvertering (A/D).....	47
Figure 6, A/D praktisk	48
Figure 7, hysteresis sløjfe.....	49
Figure 8, diagram for Random Statemachine	52
Figure 9, modstand i Serieforbindelse	66
Figure 10, modstand i Parallelforbindelse.....	66
Figure 11, modstand i Serie- og Parallelforbindelse	67
Figure 12, ohms love trekanter	67
Figure 13, oversigt over ohms lov figur.....	68
Figure 14, præfiks	68
Figure 15, huskefigur: Dæk det, du søger, til med fingeren. De to andre bogstaver viser, om du skal gange eller dividere.....	70
Figure 16, modstand i serieforbindelse.....	73
Figure 17, modstand i parallelforbindelse	74
Figure 18, figur a, Serieforbindelse inde i en parallelforbindelse	75
Figure 19, figur b, parallelforbindelse inde i en serieforbindelse	76
figure 20, 2 modstande i serie, hvor 1 er sat i parallel.....	78
figure 21, 2 serieforbindelser sat i parallel med hinanden.....	80
Figure 22: opsætning til måling af en enkelt modstand: sort i COM, rød i Ω/V , drejeknap på Ω	82
Figure 23, tre modstande i serie. Mål mellem det yderste punkt i hver ende (A og B).	83
Figure 24, sådan kan det se ud fysisk på et breadboard: modstandene sidder på række.	84
Figure 25, sådan kan det se ud fysisk på et breadboard 2	84
Figure 26, to modstande i parallel. De grønne punkter er de fælles knuder, mål mellem probernes yderpunkter.....	85
Figure 27, På breadboard: begge modstandes ben sidder i de samme to rækker.....	86
Figure 28, parallelforbindelse fysisk	86
Figure 29, To modstande i parallel (R_1 , R_2), efterfulgt af en modstand i serie (R_{serie}).....	87
Figure 30, på breadboard, serie- og parallelforbindelse	88
Figure 31, på breadboard, serie- og parallelforbindelse	88
Figure 32, Opsætning til strømmåling: sort i COM, rød i mA-porten, drejeknap på A.	90
Figure 33, Multimeteret (ampereteret) sidder i serie, strømmen løber gennem det, ikke udenom.....	91

Figure 34, På breadboard: multimeteret indsættes midt i ledningsvejen, ikke ved siden af kredsløbet.	91
---	----

Historisk tidslinje

År	Hændelse
1971	Intel 4004: verdens første mikroprocessor, ca. 175 kHz.
1974-1976	Intel 8080 og 8085 — tidlige 8-bit processorer, 3–6 MHz, 8-bit databus, 16-bit adressebus. Zilog Z80 — konkurrent til 8085, brugt i mange klassiske hjemmecomputere.
2000	Intel Pentium 4 — ca. 1,8 GHz.
2026	AMD/Intel over 3 GHz, mange kerner.

MOORES LOV

Antallet af transistorer på en chip fordobles ca. hvert 2. år. Det er en empirisk observation (baseret på historiske data) – ikke en fysisk lov som tyngdekraften.

Chip-fabrikanter (producerer/støber chippene)

TSMC (Taiwan) og Samsung (Sydkorea) dominerer markedet.

Chip-leverandører 2021 (efter salg)

Samsung, Intel, SK Hynix, Micron, Qualcomm, Nvidia, Broadcom, Texas Instruments, MediaTek og AMD.

Praktisk tjekliste

- ☐ **RYD OP EFTER DIG:** Skriv `try / except KeyboardInterrupt`: rundt om hovedløkken og nulstil LED/PWM i except-blokken. Det går igen i alle eksempler, så det forventes formentlig igen.
- ☐ **PULL_UP = TRYK ER 0:** Den hyppigste fejl i knap-opgaver. Skriv det på hånden, hvis du er nervøs for at glemme det.
- ☐ **DIGITAL VS. PWM:** Brug `led.on()` / `off()` til ren tænd/sluk. Brug `pwm.duty_u16(...)` når opgaven nævner "lysstyrke" eller "dæmpe".
- ☐ **TEGN FØR DU KODER:** Ved statemachine-opgaver: tegn cirkler og pile først. Hver cirkel = én funktion i koden.
- ☐ **TEST ADC MED POTMETER:** Test din `u16`→volt-formel med et potmeter, før du stoler på en rigtig sensor. Hurtigste måde at opdage fejl i formlen.
- ☐ **65535, IKKE 4095:** MicroPythons `read_u16()` skalerer altid op til 16-bit, selvom ADC'en fysisk er 12-bit. Del derfor altid med 65535.

Multimeter og voltmeter

Et multimeter kan måle tre ting:

Måling	Beskrivelse
Spænding	Potentialeforskel mellem to punkter.
Strøm	Hvor meget strøm der løber gennem kredsløbet.
Modstand	Bruges til at teste komponenter/kredsløb.

Et voltmeter bygget med Pico'en er i praksis: ADC + en formel + en måde at vise resultatet på (konsol-print, LEDbargraf eller 7-segment)

Eksempel: simpelt konsol-voltmeter

```
from machine import ADC, Pin
from time import sleep

adc = ADC(Pin(26))

try:
    while True:
        adc_voltage = adc.read_u16() / 65535 * 3.3
        print(f"Spænding: {adc_voltage:.2f} V")
        sleep(0.5)
except KeyboardInterrupt:
    print("Stoppet.")
```

Hardware: Pico 2 W og printet

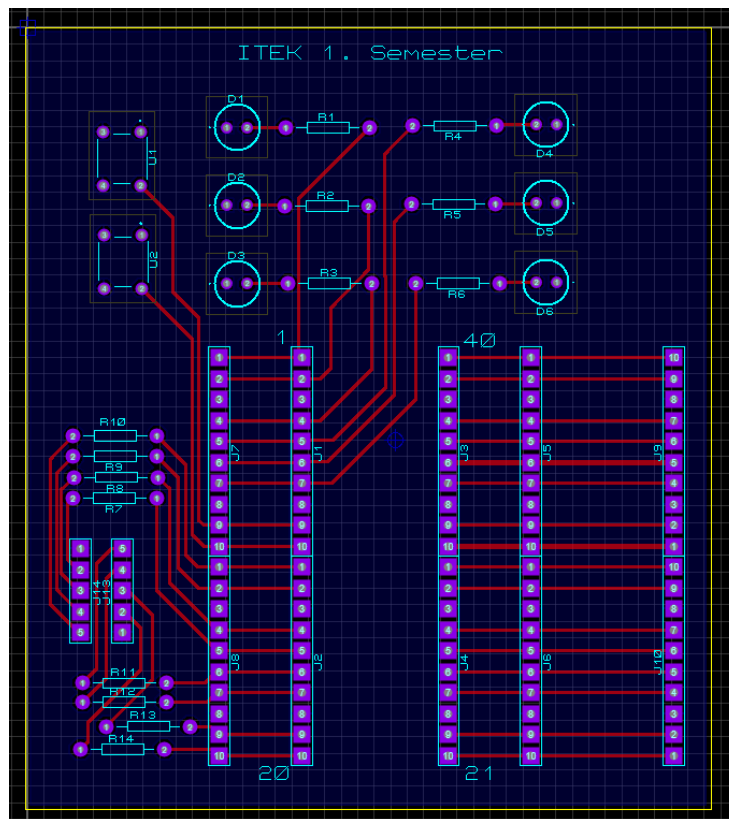


Figure 1, diagram over BBB

Raspberry Pi Pico 2 W — nøgletal

Egenskab	Beskrivelse
Chip	RP2350, dual-core (Arm Cortex-M33 @ 150 MHz eller valgfrit Hazard3 RISC-V)
Hukommelse	520 kB SRAM, 4 MB flash
Trådløs	2,4 GHz WiFi 802.11n + Bluetooth 5.2 (Infineon CYW43439-chip)
I/O	26 multifunktions-GPIO-ben, 3×12-bit ADC, SWD-debugport, TrustZone

VÆR OPMÆRKSOM

Pico 2 W kører MicroPython direkte på "bare metal" – ligesom den oprindelige Pico. ADC'en er fysisk 12-bit, men read_u16() skalerer altid op til 16-bit (0-65535) – derfor deler man med 65535, ikke 4095, i koden.

Antal	Del
1	Raspberry Pi Pico 2 W
2	Rød LED-indikator (Würth Elektronik 151051RS11000)
2	Gul LED-indikator (Würth Elektronik 151051YS04000)
2	Grøn LED-indikator (Würth Elektronik 151051VS04000)
4	330 Ω modstand (Yageo MFR-25FRF52-330R)
2	220 Ω modstand (KOA Speer CF1-2CT52R221J)
2	Trykknop (Adafruit 367)
2	Phoenix Contact MKDS 1/10-3,5-serien printklemme

VÆR OPMÆRKSOM

330 Ω bruges typisk til de 4 trafiklys-LED'er, mens 220 Ω passer bedre til 7-segment-displayets segmenter. Lavere modstand giver mere strøm/lys pr. segment – nødvendigt fordi hvert segment er en lille LED, der skal være tydeligt synlig på afstand.

Pin-layout for LED'erne

General-purpose input/output (GPIO) for led'erne på Big Beautiful Board.

Variable	GPIO	Beskrivelse
ledRL	0	Rød led venstre
ledYL	1	Gul led venstre
ledGL	2	Grøn led venstre
ledRR	3	Rød led højre
ledYR	4	Gul led højre
ledGR	5	Grøn led højre

```
ledRL = Pin (0, Pin.OUT)
ledYL = Pin (1, Pin.OUT)
ledGL = Pin (2, Pin.OUT)
ledRR = Pin (3, Pin.OUT)
ledYR = Pin (4, Pin.OUT)
ledGR = Pin (5, Pin.OUT)
```

KILDE: Big-Beautiful-Board repository (github)

Læs mere om pin-lay out, samt flere kode eksempler på Big-Beautiful-Board repository.
<https://github.com/CN-Thrige/Big-Beautiful-Board>

Pin-layout for Knapperne

Når en knap er tilsluttet et GPIO-ben på en mikrocontroller, skal benet vide, hvad det skal "falde tilbage til", når knappen ikke er trykket. Det er det, pull-up og pull-down klarer.

Egenskab	Beskrivelse
Pull-up	Et ben på μ C'en bliver trukket op til 3,3 V
Pull-down	Et ben på μ C'en bliver trukket til stel (0 V)

VÆR OPMÆRKSOM

Uden pull-up eller pull-down kan et ben "flyde" (floating) – det ved ikke, om det skal læses som 0 eller 1, når knappen ikke er trykket, og kan derfor skifte tilstand tilfældigt. Pull-up/pull-down sikrer, at benet altid har en defineret tilstand, når kredsløbet står åbent.

Sådan sættes det i MicroPython

General-purpose input/output (GPIO) for knapperne på Big Beautiful Board.

Variable	GPIO	Beskrivelse
Btn1	6	Fysisk knap
Btn2	7	Fysisk knap

Kodeblok for knapperne med PULL_UP.

```
Btn1 = Pin (6, Pin.IN, Pin.PULL_UP)
Btn2 = Pin (7, Pin.IN, Pin.PULL_UP)
```

PULL_UP: knappen forbundet til stel

```
Knap = Pin(2, Pin.IN, Pin.PULL_UP)
```

Bruges, når knappen er tilsluttet stel på Pico'en i den ene side, og en GP-port i den anden side (sådan er knappen på jeres print forbundet).

Knap-tilstand	MCU-indgang
S1 ikke sluttet (ikke trykket)	Vcc → logisk 1
S1 sluttet (trykket)	Stel → logisk 0

PULL_UP: knappen forbundet til 3,3 V

```
Knap = Pin(2, Pin.IN, Pin.PULL_DOWN)
```

Bruges, når knappen i stedet er tilsluttet **3,3 V** på Pico'en.

Knap-tilstand	MCU-indgang
S1 ikke sluttet (ikke trykket)	Vcc → logisk 1
S1 sluttet (trykket)	Stel → logisk 0

VÆR OPMÆRKSOM

De to opsætninger giver modsat logik. Med PULL_UP er tryk = 0. Med PULL_DOWN er tryk = 1. Tjek altid, hvordan knappen faktisk er forbundet på dit print, før du antager hvilken af de to du skal bruge.

KILDE: Pull up and DOWN.pptx

Læs mere i Pull up and DOWN.pptx

Pin-layout for 7-segment display

Variable	GPIO	Position	Segm.
Led1	8	bund	d
Led2	9	bund-venstre	e
Led3	10	midt	g
Led4	11	top-venstre	f
Led5	12	top	a
Led6	13	top-højre	b
Led7	14	bund-højre	c
Led8	15	punktum	dp

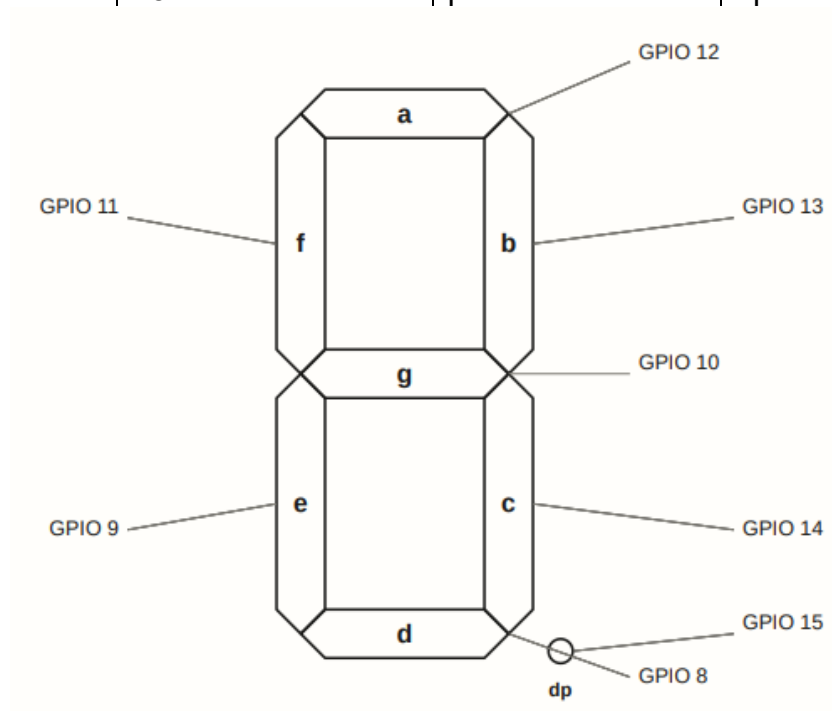


Figure 2, 7 segment display with GPIO

Pin-layout for ADC

ADC-indgangen (potmeter/spænding) sidder fast på Pin (26) (ADC0).

```
from machine import ADC
from sleep import sleep

pot = ADC (Pin (26))
```

VÆR OPMÆRKSOM

Pin.PULL_UP betyder, at knappen læses som 0, når den er trykket ned – ikke 1. Tænk det som "normalt trukket op til 1 (sluppet), tryk trækker ned til 0".

Oversigt over Big Beautiful Board repository exam kode

Koden kan findes her: <https://github.com/CN-Thrige/Big-Beautiful-Board/tree/main/Exam>

A/D, ADC OG Batteri-spændingsindikator programmer

battery_voltage_indicator.py

Titel: Batteri-spændingsindikator med tre faste tærskler

Hvad programmet indeholder:

Læser en spænding via ADC(Pin(26)) og styrer tre almindelige digitale output-pins (grøn, gul, rød) som simple on/off-LED'er (ikke PWM). read_voltage() omregner den rå ADC-værdi (0-65535) til en spænding 0-3,3 V og ganger den med en fast SCALE-faktor for at simulere en spændingsdeler, så den reelle batterispænding (op til fx 12 V) kan udledes fra den lavere ADC-spænding. I hovedløkken slukkes alle LED'er (all_off()), og derefter tændes præcis én LED ud fra hardkodede tærskler: grøn ved ≥ 10 V, gul ved ≥ 6 V, rød ved ≥ 3 V. Er spændingen under 3 V, blinker den røde LED (on/off med 0,2 s mellemrum) som advarsel, og continue springer resten af løkken over.

Hvad kan det bruges til:

Praktisk eksempel på at bruge ADC-input til at overvåge en spænding (fx et batteri) og give visuel status via tre-trins LED-indikering samt en ekstra blinkende 'lav batteri'-advarsel - god øvelse i spændingsdeler-omregning og simple if/elif-tærskelsammenligninger.

battery_voltage_getint_style.py

Titel: Batteri-spændingsindikator med brugerdefinerede tærskler

Hvad programmet indeholder:

Samme grundopbygning som den faste version (ADC på pin 26, tre LED'er på pin 0-2, samme SCALE-omregning og read_voltage()), men tærsklerne for grøn, gul og rød er ikke hardkodede. I stedet spørges brugeren ved opstart om alle tre tærskler via get_int() (samme validerede input-mønster som i PWM- og elevator-programmerne). Hovedløkken er identisk i logikken (all_off(), if/elif på v mod de tre tærskler, blink ved meget lav spænding), men bruger nu de brugervalgte grænseværdier. Hele løkken er desuden pakket i try/except KeyboardInterrupt, så alle LED'er slukkes pænt ved afbrydelse.

Hvad kan det bruges til:

Viser hvordan et fast tærskel-baseret indikatorprogram kan gøres konfigurerbart ved opstart, så det samme program kan tilpasses forskellige batterityper/spændingsniveauer uden at ændre koden - samt korrekt oprydning af output ved Ctrl+C.

Tidligere eksamensbesvarelser

20_points.py

Titel: Simpel knap-styret to-LED-sekvens (20 point)

Hvad programmet indeholder:

Meget kort program uden PWM: to LED'er (p1 = rød på pin 0, p0 = gul på pin 1) styres som almindelige digitale outputs, og én knap (button1 på pin 6, PULL_UP) læses i en uendelig while-løkke. Når knappen trykkes (værdi 0), tændes gul i 0,5 sekund og slukkes, derefter tændes rød i 0,5 sekund og slukkes - en fast, blokerende sekvens uden debounce, uden try/except og uden PWM overhovedet.

Hvad kan det bruges til:

Eksamensbesvarelse der viser et minimalt, fungerende udgangspunkt: knap-læsning og digital LED-styring virker, men opgaven om PWM-baseret lysstyrke/blink er ikke løst - kun grundlæggende Pin.IN/Pin.OUT og sleep()-baseret sekventiel styring er demonstreret. Gode til at sammenligne med de fulde PWM-løsninger og se, hvad der typisk mangler for fuld pointscore (PWM-objekter, duty cycle, frekvensvalg, oprydning).

incomplete_30_points_worth_15points.py

Titel: PWM-brightness/blink-forsøg med fejl i frekvensbrug (15 af 30 point)

Hvad programmet indeholder:

Forsøger at genskabe brightness+blink-opgaven med to PWM-LED'er (rød på pin 0, gul på pin 1), men har flere problemer: den bruger machine.Pin(...) uden at have importeret machine-modulet selv (kun Pin og PWM er importeret via 'from machine import Pin, PWM'), hvilket vil give en fejl ved kørsel. Endnu vigtigere er, at PWM-frekvensen (freq()) sættes til brugerens ønskede blink-frekvens (0-20 Hz) i stedet for en fast, høj carrier-frekvens (fx 1000 Hz) til selve dæmpningen - det er den centrale misforståelse mellem PWM_CARRIER og synligt blink, som er beskrevet i eksamensnoterne. I stedet for et rent on/off-blink rammer koden en for-løkke, der glider duty_cycle op og ned fra 0 til 65535 med et brugerdefineret trin (duty_step) - det giver reelt en fade-effekt, ikke den ønskede kombination af fast lysstyrke og separat blink-frekvens.

Hvad kan det bruges til:

God 'lærings-case' fra en rigtig eksamen: viser to klassiske faldgruber - (1) at blande PWM-carrier-frekvensen sammen med den ønskede synlige blink-frekvens, og (2) at bruge et ikke-importeret modulnavn (machine.Pin frem for blot Pin). Velegnet til at diskutere, hvorfor programmet kun gav delvis pointscore, og hvordan det kunne rettes op til den fulde, korrekte løsning (se fixed_30_points_pwm.py).

fixed_30_points_pwm.py

Titel: Korrekt PWM-brightness/blink-løsning (fuldt 30 point)

Hvad programmet indeholder:

Den rettede/fulde udgave af samme opgave som ovenfor: to PWM-LED'er (rød, gul) får en fast PWM_CARRIER på 1000 Hz til selve dæmpningen, adskilt fra brugerens valgte blink-frekvens (0-20 Hz) og lysstyrke (0-100%), indhentet via den validerede get_int(). duty beregnes én gang ud fra lysstyrken, og set_leds() tænder/slukker begge LED'er til denne faste duty. Er frekvensen 0, lyser LED'erne solidt; ellers veksler et while-loop mellem tændt og slukket med half_period-pauser. Programmet afsluttes korrekt med duty sat til 0 og deinit() på begge PWM-objekter ved Ctrl+C.

Hvad kan det bruges til:

Reference-facit til de to ufuldstændige besvarelser ovenfor: viser den korrekte adskillelse af PWM_CARRIER (dæmpning) og brugerstyret blink-frekvens, korrekt modulimport og pæn oprydning - alt det, der mangler i 15-points- og 20-points-versionerne. God til at gennemgå trin for trin, hvad der adskiller en delvis fra en fuld eksamensbesvarelse.

Trafiklys- og lyskryds-programmer

traffic_light_single.py

Titel: Trafiklys med fast timing (state machine, digitalt output)

Hvad programmet indeholder:

Tre almindelige digitale outputs (rød, gul, grøn på pin 0-2) styres af en state machine bygget som funktioner, ligesom i elevator-programmerne: state_green(), state_yellow() og state_red() slukker først alle LED'er (all_off()), tænder deres egen farve, venter en hardkodet varighed med sleep() og returnerer navnet på næste tilstands-funktion. Det yderste while-loop (state = state()) driver cyklussen grøn -> gul -> rød -> grøn i en uendelig løkke med faste tider (5,5 s grøn/rød, 1,5 s gul).

Hvad kan det bruges til:

Simplest mulige trafiklys-eksempel: viser samme state-machine-mønster som elevator-programmerne, men anvendt på en cyklisk (ikke lineær) sekvens uden brugerinput eller fejlhåndtering - godt udgangspunkt før man tilføjer konfigurerbare tider eller flere retninger.

traffic_light_getint_style.py

Titel: Trafiklys med brugerdefinerede tider (get_int-stil)

Hvad programmet indeholder:

Samme state-machine-struktur som den faste version (state_green/state_yellow/state_red, all_off()), tre digitale LED'er), men de tre tidsintervaller (green_time, yellow_time, red_time) hentes ved opstart via get_int() med individuelle gyldige intervaller (fx grøn 1-30 s, gul 1-10 s). Hele kørslen er desuden pakket i try/except KeyboardInterrupt, så alle LED'er slukkes pænt ved Ctrl+C.

Hvad kan det bruges til:

Viser hvordan det faste trafiklys gøres konfigurerbart uden at ændre selve state-machine-logikken - kun tidskonstanterne erstattes af brugerinput - samt korrekt oprydning ved afbrydelse, som den simple version mangler.

traffic_light_typed_timing.py

Titel: Trafiklys med brugerdefinerede tider (variant)

Hvad programmet indeholder:

Funktionelt identisk med traffic_light_getint_style.py: samme tre PWM-uafhængige digitale LED'er, samme get_int()-baserede indhentning af green_time/yellow_time/red_time, samme state-machine-funktioner (state_green/state_yellow/state_red) og samme try/except-oprydning ved Ctrl+C. Eneste forskel er en mindre ændring i afsluttende print-tekst ('Stopped.' i stedet for 'Stopped, outputs cleared.').

Hvad kan det bruges til:

Endnu en variant/iteration af det konfigurerbare trafiklys - nyttig som sammenligningseksempel for at se, hvor lidt der reelt skal ændres mellem to 'næsten identiske' indleveringer, og hvad der typisk er kosmetiske forskelle versus funktionelle forskelle.

lyskryds_intersection.py

Titel: Fuldt lyskryds med to retninger og fast timing (state machine)

Hvad programmet indeholder:

Udvider trafiklys-idéen til et helt vejkryds med to retninger (nord-syd 'ns' og øst-vest 'ev'), hver med rød/gul/grøn på egne pins (0-2 og 3-5). `set_lights(ns, ev)` sætter alle seks output-pins ud fra to tuples af 0/1-værdier (fx (1,0,0) = kun rød). State maskinen har otte tilstande: `all_red` (begge retninger røde, sikkerhedspause) -> `ns_ready` (NS får gul 'gør klar') -> `ns_go` (NS grøn, EV forbliver rød) -> `ns_stop` (NS gul) -> `all_red2` -> `ev_ready` -> `ev_go` -> `ev_stop` -> tilbage til `all_red`. Hver tilstand venter en hardkodet varighed og returnerer næste tilstands-funktion, på samme måde som i de simple trafiklys-programmer.

Hvad kan det bruges til:

Realistisk eksempel på en mere kompleks tilstandsmaskine, der koordinerer to modstridende trafikstrømme sikkert (med all-red-sikkerhedspauser mellem hver retnings grønne fase) - god øvelse i at udvide et simpelt state-machine-mønster til flere samtidige lyssæt uden at bruge PWM.

Lyskryds_getint_style.py

Titel: Fuldt lyskryds med to retninger og brugerdefinerede tider

Hvad programmet indeholder:

Samme otte-tilstands lyskryds-struktur som `lyskryds_intersection.py` (`all_red`, `ns_ready`, `ns_go`, `ns_stop`, `all_red2`, `ev_ready`, `ev_go`, `ev_stop`) og samme `set_lights()`-funktion til at styre begge retningers rød/gul/grøn. Forskellen er, at grøn tid, gul tid og all-red-sikkerhedstid nu hentes fra brugeren via `get_int()` (grøn 1-30 s, gul 1-10 s, all-red 1-5 s) i stedet for at være hardkodet, og bruges konsekvent i alle otte tilstandsfunktioner. En ekstra `all_off()`-funktion og `try/except KeyboardInterrupt` sikrer, at alle seks LED'er slukkes korrekt ved afbrydelse.

Hvad kan det bruges til:

Kombinerer de to tidligere idéer - fuldt to-retnings-lyskryds og brugerkonfigurerbare tider - i én løsning. God til at diskutere, hvordan man parametriserer en større, multi-tilstands state machine uden at gentage tidskonstanter flere steder, samt vigtigheden af en fælles all-red-fase som sikkerhedsmekanisme mellem to trafikretninger.

Statemachine

trafiklys_enkelt.py

Titel: Trafiklys, ét lyssæt med brugerdefinerede tider (opgave 01)

Hvad programmet indeholder:

Grundlæggende trafiklys-state machine (grøn -> gul -> rød -> grøn) bygget med funktioner der returnerer næste state, som i de øvrige trafiklys-programmer. Her styres dog både 'venstre' og 'højre' sæt LED'er (ledGL/ledGR osv.) samtidig med samme farve, selvom kun ét lyssæt reelt bruges i opgaven. Tiderne for grøn, gul og rød hentes fra brugeren via `get_int()` ved opstart, og hele forløbet er pakket i try/except KeyboardInterrupt med oprydning (`alle_slukket()`).

Hvad kan det bruges til:

Fungerer som skabelon-øvelse for selve state-machine-mønstret ('en state er en funktion der returnerer næste state') anvendt på et simpelt, cyklisk, tidsstyret trafiklys - uden knapper eller sensorer, kun brugerinput ved opstart.

lyskryds_kryds_statemachine.py

Titel: Fuldt lyskryds med 6 states og brugerdefinerede tider (opgave 02)

Hvad programmet indeholder:

Implementerer et helt vejkryds (venstre/højre retning, 6 LED'er på pin 0-5) som en state machine med seks tilstande: `s0_groen_venstre` -> `s1_gul_venstre` -> `s2_alt_roedt_a` -> `s3_groen_hoejre` -> `s4_gul_hoejre` -> `s5_alt_roedt_b` -> tilbage til `s0`. Hjælpefunktionen `sæt()` sætter alle seks LED'er i én linje ud fra 0/1- værdier. Grøn tid, gul tid og alt-rødt-sikkerhedstid hentes via `get_int()` ved opstart og bruges i alle relevante states.

Hvad kan det bruges til:

Klassisk 'to trafiklys koblet sammen så de aldrig er grønne samtidig'-opgave, med indbygget alt-rødt-sikkerhedspause mellem retningsskiftene - fuldt konfigurerbar version uden kaldeknapper (se `lyskryds_med_knapper_statemachine.py` for udvidelsen med det).

lyskryds_med_knapper_statemachine.py

Titel: Lyskryds med kaldeknapper der kan afkorte grøn fase (opgave 03)

Hvad programmet indeholder:

Udvider det seks-tilstands lyskryds med to kaldeknapper (`button1/button2`): den grønne fase kører altid en minimumstid (`min_groen_tid`), men kan derefter afkortes, hvis den modsatte retnings knap trykkes, i stedet for altid at vente til `maks_groen_tid`. Funktionen `groen_med_afkortning()` håndterer dette ved først at sove `min_groen_tid` og derefter polle knappen i 0,1 s-intervaller op til `maks_groen_tid`. Min/maks grøn tid, gul tid og alt-rødt-tid er alle brugerkonfigurerbare via `get_int()`.

Hvad kan det bruges til:

Viser hvordan en tidsstyret state machine kan gøres reaktiv over for input (kaldeknapper), samtidig med at et minimumskrav (min. grøn-tid) altid overholdes af sikkerhedshensyn - relevant eksamensdiskussion om at kombinere timer-baserede og knap-baserede overgange i samme state.

statemachine_trafiklys_enkelt.py

Titel: Trafiklys-skabelon med fast timing (dokumenteret state machine-mønster)

Hvad programmet indeholder:

En ren, kommenteret skabelonversion af trafiklys-state maskinen: tre LED'er (grøn/gul/rød på pin 2-4), tre state-funktioner (state_green/state_yellow/state_red) med hardkodede tider (5,5 s / 1,5 s / 5,5 s), og en all_off()- hjælpefunktion. Koden er bevidst holdt simpel og velkommenteret som eksempel på selve mønstret: 'state = state(); hver state returnerer næste state'.

Hvad kan det bruges til:

Beregnet som den grundskabelon man kan genbruge til alle andre state-machine- opgaver (elevator, lyskryds, termostat osv.) - selve pointen er strukturen, ikke funktionaliteten. God at have som opslag til eksamen for det generelle state-machine-mønster.

statemachine_lyskryds_kryds.py

Titel: Lyskryds Nord-Syd/Øst-Vest med fast timing (6 states)

Hvad programmet indeholder:

Samme grundidé som lyskryds_kryds_statemachine.py, men navngivet efter reelle retninger (Nord-Syd 'ns' og Øst-Vest 'ew', LED'er på pin 2-7) og med hardkodede tider (GREEN_TIME=5.0, YELLOW_TIME=1.5, ALL_RED_TIME=1.0) i stedet for brugerinput. Seks states (state_ns_green -> state_ns_yellow -> state_all_red_1 -> state_ew_green -> state_ew_yellow -> state_all_red_2 -> tilbage) styres via to hjælpefunktioner set_ns()/set_ew(), der hver sætter tre LED'er ud fra rød/gul/grøn-parametre.

Hvad kan det bruges til:

Alternativ, mere 'produktionsnær' navngivning af det samme lyskrydsmønster - nyttig til at øve at genkende samme underliggende state machine under forskellige variabel- og pin-navne, hvilket ofte sker mellem opgaveversioner til eksamen.

statemachine_elevator.py

Titel: Elevator med 3 etager og etage-kaldeknapper

Hvad programmet indeholder:

Simulerer en elevator med tre etager. Tre knapper (GP6-8) fungerer som etage-kald, og tre LED'er (GP2-4) viser, hvilken etage kabinen aktuelt er på (`show_current_floor()`). To states driver det hele: `state_idle()` venter på et knaptryk og sætter `target_floor`, mens `state_moving()` flytter `current_floor` ét skridt (1 sekund pr. etage) ad gangen mod `target_floor`, indtil de matcher, hvorefter den returnerer til `state_idle`. En lille `sleep(0.1)` i hovedløkken forhindrer for hyppig knap-aflæsning.

Hvad kan det bruges til:

Mere realistisk elevator-eksempel end de tidligere to-etagers versioner: demonstrerer en state machine med intern 'position vs. mål'-logik, hvor samme state (`state_moving`) gentagne gange kalder sig selv, indtil en betingelse er opfyldt - et nyttigt mønster for gradvis bevægelse/nærmer-sig-mål-opgaver.

`statemachine_fake_thermostat.py`

Titel: Hysterese-termostat med simuleret temperatur (opgave 11, bonus)

Hvad programmet indeholder:

Simulerer en varmestyring med hysterese: et potentiometer på ADC (GP26) bruges som 'fup-temperatur' (0-3,3V mappes til 0-50 grader via `laes_temperatur()`). En boolean `varme_er_tændt` tændes, når temperaturen kommer ned på `TAENDT_GRAENSE` (23°) eller under, og slukkes først når den når op på `SLUKKET_GRAENSE` (25°) eller over - i det mellemliggende bånd (23-25°) forbliver tilstanden uændret, hvilket er selve hysteresen. Én LED (`ledRL`) viser 'varme tændt'-status, og temperatur/status printes hvert 0,5 sekund.

Hvad kan det bruges til:

Introducerer hysterese-princippet: undgår at et varmelegeme/en afkøler 'flimrer' tændt/slukket omkring ét enkelt sætpunkt ved at bruge to forskellige tærskler for hhv. tænd og sluk. Simpel, løkkebaseret (ikke funktions-state- baseret) implementering af konceptet.

`statemachine_hysterese_thermostat.py`

Titel: Hysterese-termostat, uddybet version med `read_temperature()`

Hvad programmet indeholder:

Funktionelt samme hysterese-logik som den 'fake' termostat-version (`ON_TEMP=23`, `OFF_TEMP=25`, simuleret 0-50°C via et potentiometer på ADC0), men med en dedikeret `read_temperature()`-funktion, der eksplicit regner rå ADC-værdi om til spænding og derefter til grader i to trin, samt mere beskrivende variabelnavngivning (`heater`, `heater_on`, status-tekst 'TÆNDT'/'slukket'). Koden er kommenteret med en forklaring af, hvorfor

hysteresis er nødvendig, og en note om at kun omregningsformlen skal ændres, hvis potentiometeret erstattes af en rigtig temperatursensor.

Hvad kan det bruges til:

Uddybende/didaktisk variant af samme hysteresis-termostat - god som den version man forklarer princippet ud fra til eksamen, da den har den tydeligste adskillelse mellem sensor-aflæsning, temperaturberegning og selve hysteresis-beslutningen.

`tilfældig_lyskryds_statemachine.py`

Titel: 'Random machine', tilfældigt valgt næste state (opgave 12, bonus)

Hvad programmet indeholder:

Demonstrerer, at en state ikke behøver gå til en fast, forudbestemt næste state. Alle seks lyskryds-LED'er lægges i en liste (`alle_leds`), og den eneste state, `tilfældig_state()`, slukker alt, vælger én tilfældig LED med `random.choice()`, tænder den, venter 0,5 sekund og returnerer sig selv som næste state - så løkken fortsætter med nye tilfældige valg i det uendelige. Koden er tydeligt markeret som en konceptdemonstration og IKKE en gyldig trafikstyring, da der ingen sikkerhed er mod at flere retninger er 'grønne' samtidig.

Hvad kan det bruges til:

Ren konceptøvelse: viser at 'næste state' kan være ikke-deterministisk (afhænger af `random()` i stedet for tid/input), nyttig pointe til eksamen om, at state machine-mønstret er mere generelt end blot faste sekvenser.

`random_statemachine_1_.py`

Titel: Tilfældig state machine med 3 LED-states og mulig stop-tilstand

Hvad programmet indeholder:

Tre LED'er (`led0-led2`) styres af tre states (`state0`, `state1`, `state2`), der hver tænder sin egen LED, venter 1 sekund og derefter vælger næste state delvist tilfældigt: `state0` går til `state1` eller `state2` (50/50 med `random()`), `state1` går til `state0` eller `state2`, og `state2` har 50% chance for at returnere `None` - hvilket får `while state:`-løkken til at stoppe helt, da `None` er 'falsk'. Efter løkken kaldes `all_off()` og der printes 'Stop'.

Hvad kan det bruges til:

Eksempel på en tilfældig state machine, der (i modsætning til `tilfældig_lyskryds_statemachine.py`) faktisk kan afslutte sig selv - viser hvordan man bruger en state's returværdi `None` som en eksplicit 'slut'-tilstand i mønstret `'while state: state = state()'`.

random_statemachine_random_getint_style.py

Titel: Tilfældig state machine med konfigurerbar tid og stop-sandsynlighed

Hvad programmet indeholder:

Samme tre-state, tilfældigt-forgrenende struktur som random_statemachine(1).py (state0/state1/state2, samme 50/50-forgrening mellem state0 og state1), men brugeren angiver ved opstart via get_int() hvor lang tid hver state varer (step_time) og hvor stor chance (i procent) der er for, at state2 stopper maskinen (stop_chance_pct, omregnet til stop_threshold 0-1). Hele forløbet er desuden pakket i try/except KeyboardInterrupt for ekstra oprydning, ud over den naturlige afslutning når state2 returnerer None.

Hvad kan det bruges til:

Gør den tilfældige state machine konfigurerbar og eksperimenterbar - man kan fx sætte stop-chancen til 100% for altid at stoppe efter state2, eller 0% for at den aldrig stopper af sig selv - god til at diskutere sandsynlighedsbaserede overgange og hvordan de kan parametriseres.

pwm_fysisk_knap_lysstyrke.py

Titel: PWM-lysstyrke styret med fysiske op/ned-knapper (opgave 05)

Hvad programmet indeholder:

Direkte svar på opgaven fra PWM-slides om at 'bruge knapperne til at skrue op og ned for LED'en': to knapper (button1 = op, button2 = ned, begge PULL_UP) og én PWM-LED (pin 2) med fast PWM_CARRIER på 1000 Hz. brightness_pct starter på 50% og justeres i trin af TRIN (5%) ad gangen, når den relevante knap er trykket, med en simpel sleep(0.2)-debounce. Den nye duty_u16-værdi og procent printes ved hver ændring.

Hvad kan det bruges til:

Samme grundfunktion som pwm_buttons.py fra PWM-kapitlet, men med danske kommentarer/print og direkte kobling til den konkrete slide-opgave - god reference for netop den formulering af opgaven til eksamen ('brug knapperne til at skrue op og ned').

adc_spændingsmåler.py

Titel: Simpelt voltmeter med ADC (opgave 06)

Hvad programmet indeholder:

Læser en analog spænding via ADC(Pin(26)) og omregner den rå værdi (0-65535) til volt (0-3,3V) med formlen $spænding = pot_value / 65535 * 3.3$, som beskrevet i AD-konverter-

slides. Måler og printer værdien 10 gange i sekundet (`sleep(0.1)`) i en uendelig løkke, indtil Ctrl+C.

Hvad kan det bruges til:

Det mest grundlæggende ADC-eksempel: viser sammenhængen mellem Pico'ens digitale ADC-værdi (0-65535) og den tilsvarende analoge spænding (0-3,3V) uden yderligere logik - byggestenen for alle de øvrige spændings-/sensor-programmer.

`adc_batteri_spændingsdeler_statemachine.py`

Titel: Batterispænding via spændingsdeler med advarsel (opgave 07)

Hvad programmet indeholder:

Udvider det simple voltmeter til at måle en batterispænding (6,4-8,4V), der er for høj til at måles direkte af Pico'ens ADC (0-3,3V). En fysisk spændingsdeler ($R1/R2$) sænker spændingen ned i måleområdet, og koden ganger målingen op igen med $SKALA_FAKTOR = (R1+R2)/R2$ for at vise den reelle batterispænding. Hvis den beregnede batterispænding kommer under 6,4V, printes en advarsel ud over den almindelige måling, som opdateres hvert 0,5 sekund.

Hvad kan det bruges til:

Praktisk anvendelse af ADC til overvågning af en spænding, der ligger uden for Pico'ens direkte måleområde, viser princippet bag spændingsdelere i kode (omregning frem og tilbage) samt simpel tærskel-baseret advarselslogik.

`analogt_voltmeter_led_bar.py`

Titel: 'Analogt' voltmeter som LED-bar med kritisk blink (opgave 08)

Hvad programmet indeholder:

Måler en batterispænding (0-12V, via spændingsdeler og $SKALA_FAKTOR$ ligesom i opgave 07) og viser den som en tre-trins LED-bar (grøn/gul/rød på pin 0-2) i stedet for et tal: grøn ved $\geq 10V$, gul ved $\geq 6V$, rød ved $\geq 3V$, og ved kritisk lav spænding (under 3V) blinker den røde LED ved at vende en `blink_tilstand`- boolean hvert loop-gennemløb (hver 0,5 s). Spændingen printes desuden løbende til konsollen.

Hvad kan det bruges til:

Kombinerer ADC-måling og tærskel-baseret LED-indikering (som batteri- kapitlets programmer) med en ekstra 'kritisk lav'-tilstand, der bruger blink i stedet for konstant lys som ekstra visuel advarsel - god øvelse i at lægge et diskret alarmniveau oven på et kontinuert måleområde.

7segment_nedtælling_statemachine.py

Titel: 7-segment nedtælling / 'raket-affyring' (opgave 09)

Hvad programmet indeholder:

Styrer et 7-segment-display (segmenter a-g på GP10-16) ud fra et opslag (CIFRE-dictionary), der for hvert ciffer 0-9 angiver, hvilke segmentbogstaver der skal lyse. vis_ciffer() slår det aktuelle tal op og tænder/slukker hvert segment-pin tilsvarende (med SEGMENT_ON/SEGMENT_OFF, der kan vendes om for fælles-anode vs. fælles-katode displays). Brugeren angiver et starttal (0-9) via get_int(), og programmet tæller ned med ét sekund pr. ciffer, printer 'Affyring!' ved 0, og slukker til sidst alle segmenter.

Hvad kan det bruges til:

Praktisk eksempel på at oversætte et logisk 'display-alfabet' (hvilke segmenter udgør hvert ciffer) til konkrete pin-tændinger - en anden type output-styring end LED'er/PWM, men med samme grundprincip om at slå en tilstand op i en tabel og sætte flere outputs samtidig.

gy53_afstandsmåling_statemachine.py

Titel: Afstandsmåling med GY-53-sensor og 3 LED-zoner (opgave 10)

Hvad programmet indeholder:

Læser en pulsbredde-moduleret afstandssensor (GY-53) på en digital indgang (pin 16) ved manuelt at måle, hvor længe signalet er højt, med time.ticks_us()/ticks_diff() - en form for 'manuel PWM-aflæsning' i stedet for en indbygget PWM-læsefunktion.

maal_afstand_mm() venter på pulsens start og slutning og omregner pulslængden (μ s) til millimeter via en fast faktor (afstand = pulslængde/10). vis_zone() tænder én af tre LED'er (grøn/gul/rød) afhængigt af om afstanden er 0-10, 11-20 eller 21-30 cm; udenfor det område forbliver alle tre slukkede.

Hvad kan det bruges til:

Viser hvordan man selv kan måle et pulsbredde-signal fra en sensor uden en færdig bibliotek-funktion, og hvordan en kontinuert måling (afstand) omsættes til diskrete zoner med LED-feedback - relevant hvis eksamen spørger ind til andre input-typer end knapper og ADC.

PWM-programmer

PWM-programmer, Oversigt til eksamensforberedelse.

pwm_single_led.py

Titel: PWM-styring af én LED (blink + lysstyrke)

Hvad programmet indeholder:

Programmet opretter ét PWM-objekt på pin 0 og sætter en fast intern switching-frekvens (PWM_CARRIER = 1000 Hz), som kun bruges til dæmpning. Via input() spørges brugeren om en blink-frekvens (0-20 Hz) og en lysstyrke (0-100%), valideret af hjælpefunktionen get_int(). Lysstyrken omregnes til en duty_u16-værdi (0-65535). Funktionen set_led() tænder LED'en til den valgte duty eller slukker den helt. Er frekvensen 0, lyser LED'en solidt; ellers beregnes en half_period, og et while-loop skifter tændt/slukket med sleep() imellem. Ved Ctrl+C sættes duty til 0 og PWM-objektet deinitialiseres.

Hvad kan det bruges til:

Demonstrerer grundprincippet i PWM: forskellen på duty cycle (lysstyrke) og et software-styret blink oven på PWM-signalet. God basisøvelse til at forstå duty_u16(), freq() og korrekt oprydning med deinit().

pwm_three_leds.py

Titel: PWM-styring af tre LED'er samtidig

Hvad programmet indeholder:

Udvidelse af enkelt-LED-programmet til tre LED'er (rød, gul, grøn på pin 0-2), der alle får samme PWM_CARRIER-frekvens. Brugeren angiver én fælles blink-frekvens og lysstyrke via get_int(), og duty beregnes én gang. set_leds() sætter alle tre LED'er til samme duty (tændt) eller 0 (slukket) i samme funktionskald. Blink-logikken (frequency == 0 vs. half_period-loop) er identisk med enkelt-LED-versionen. Ved afbrydelse ryddes alle tre PWM-objekter op.

Hvad kan det bruges til:

Viser hvordan samme PWM-mønster kan genbruges og styre flere pins parallelt med ét fælles duty/blink-signal. Fx til at illustrere synkroniseret blink eller gruppelys.

pwm_two_leds_independent.py

Titel: To LED'er med uafhængig blink-frekvens og lysstyrke

Hvad programmet indeholder:

Opretter to PWM-objekter (rød og gul) og spørger separat om blink-frekvens og lysstyrke for hver af dem. I stedet for at bruge sleep() til blink (som ville blokere begge LED'er

samtidig), bruges en tidsbaseret løkke: en fælles 'tick'-variabel `t` opdateres med 0.01 s ad gangen, og hver LED har sin egen `last_toggle-tidsstempel` og `half_period`. Når tiden siden sidste skift overstiger `half_period` for den pågældende LED, vendes dens tilstand (on/off) uafhængigt af den anden LED.

Hvad kan det bruges til:

Illustrerer non-blocking timing - hvordan man styrer flere uafhængige tidsforløb i samme løkke uden at bruge `sleep()` til at blokere hele programmet. Vigtigt koncept i indlejrede systemer, hvor flere ting skal ske 'samtidig'.

`pwm_buttons.py`

Titel: Lysstyrkeregulering med to knapper (op/ned)

Hvad programmet indeholder:

To digitale indgange (pin 6 og 7) er sat op som knapper med `PULL_UP`, og én LED styres via PWM på pin 0 med fast `freq(1000)`. I hovedløkken tjekkes knappernes værdi: er `btn_up` trykket (værdi 0 pga. pull-up), øges `brightness_pct` med `STEP (5)`; er `btn_down` trykket, mindskes den. En kort `sleep(0.15)` bruges som simpel debounce. Den nye procent omregnes løbende til `duty_u16` og sendes til LED'en.

Hvad kan det bruges til:

Klassisk 'dæmper med knapper'-scenarie: viser hvordan digitale input (`Pin.IN`, `PULL_UP`) kombineres med PWM-output til manuel lysstyrkejustering, direkte svar på opgaven fra slides om at bruge knapper til at skrue op/ned.

`pwm_potentiometer_brightness.py`

Titel: Lysstyrke styret af potentiometer (ADC)

Hvad programmet indeholder:

Bruger `ADC(Pin(26))` til at læse en potentiometers analoge spænding og en LED (PWM på pin 2) til output. `read_u16()` returnerer en værdi 0-65535, som praktisk nok matcher `duty_u16()`'s interval 1:1, så værdien skrives direkte videre uden omregning. I en løkke læses potentiometeret hvert 0,1 sekund, `duty` sættes, og den tilsvarende procent udregnes og printes til konsollen.

Hvad kan det bruges til:

Demonstrerer analog input via ADC og hvordan en kontinuert fysisk drejeknap kan bruges som direkte, jævn lysstyrkekontrol, i modsætning til trin-baseret knapstyring. God til at forklare `ADC.read_u16()` og sammenhængen mellem ADC-opløsning og PWM-opløsning.

pwm_traffic_light_sequence.py

Titel: Trafiklys-sekvens med fade mellem tilstande

Hvad programmet indeholder:

Tre PWM-LED'er (rød, gul, grøn) simulerer et trafiklys. Funktionen `fade_to()` læser hver LED's nuværende duty og interpolerer lineært til en målværdi over `FADE_STEPS` trin med en lille `sleep()` mellem hvert trin, så overgangen glider blødt i stedet for at springe. Hovedløkken kører den klassiske trafiklys-cyklus: rød -> rød+gul -> grøn -> gul -> rød, med forskellige 'hold'-tider (`HOLD_RED`, `HOLD_GREEN` osv.) mellem hver fade.

Hvad kan det bruges til:

Viser et praktisk state-machine-eksempel med bløde overgange (cross-fade) mellem flere LED'er styret af PWM, og hvordan tidsstyring (hold-tider) kan kombineres med gradvise duty-ændringer for et mere realistisk visuelt resultat.

pwm_morse_sos.py

Titel: Morse-kode SOS med én LED

Hvad programmet indeholder:

En PWM-LED bruges til at sende SOS i morsekode. Grundenheden `DOT` (0,2 s) definerer et 'kort' blink, `DASH` er tre gange så langt, og der er forskellige pauser mellem symboler, bogstaver og hele ordet (`GAP_SYMBOL`, `GAP_LETTER`, `GAP_WORD`). Mønstret for S-O-S er defineret som en liste af lister med `DOT/DASH`-værdier. Funktionen `blink()` tænder LED'en fuldt, venter den angivne varighed, slukker og venter en symbol-pause. Løkken gennemløber bogstaverne og erstatter den sidste pause i hvert niveau med den relevante, længere pause.

Hvad kan det bruges til:

Kreativt eksempel på timing-baseret signalering med PWM som simpel on/off-styring (duty enten `FULL` eller `OFF`). Træner præcis brug af `sleep()` til at bygge et struktureret tidsmønster ud fra få grundlæggende tidsenheder.

pwm_simon_says.py

Titel: 'Simon Says' venstre/højre-gættelegen

Hvad programmet indeholder:

Bruger seks PWM-LED'er (tre venstre, tre højre) og to knapper. Med `urandom.getrandbits(1)` vælges tilfældigt venstre eller højre side, den tilhørende LED-gruppe blinker kort, og

spilleren skal gætte side via Btn1 (venstre) eller Btn2 (højre). `wait_for_press()` venter blokerende på et tryk, `wait_for_release()` venter til knappen slippes igen (simpel debounce/afvent). Ved korrekt gæt øges score og et nyt runde starter; ved forkert gæt nulstilles score, og `flash_all()` blinker alle LED'er som 'game over'-indikation.

Hvad kan det bruges til:

Eksempel på et lille reaktions-/hukommelsesspil, der kombinerer digitale input (knapper), PWM-output (LED-grupper) og simpel tilfældighed (`urandom`) til en komplet spil-loop med scorehåndtering.

`pwm_two_player_race.py`

Titel: To-spiller reaktionsvæddeløb

Hvad programmet indeholder:

To spillere har hver sin knap og LED (pin 2 og 5), plus en fælles 'vent'-LED (pin 0). `play_round()` tænder vent-LED'en, venter et tilfældigt interval (1-4 sekunder) via `urandom`, og tjekker undervejs om en spiller trykker for tidligt (disqualificerer sig selv). Når ventetiden er ovre, slukkes vent-LED'en, begge spiller-LED'er tændes som 'GO'-signal, og programmet venter på det første knaptryk for at kåre en vinder. `wait_for_release()` sikrer at knappen slippes før næste runde starter.

Hvad kan det bruges til:

Demonstrerer et konkurrence-/reaktionsspil med to samtidige input-kilder, tilfældig ventetid og simpel 'false start'-detektion, relevant for at forstå polling af flere knapper og tidsstyret spillogik.

`pwm_wave_breathing.py`

Titel: Bølge-/åndedrætseffekt med sinuskurve på seks LED'er

Hvad programmet indeholder:

Seks PWM-LED'er får hver en fasesforskudt sinusværdi beregnet med `math.sin()`. En fælles vinkel (angle) øges løbende, og hver LED lægger et fast fase-offset (`PHASE_STEP`) til vinklen ud fra sit indeks, så de ikke lyser i takt. Sinusværdien (-1 til 1) skales om til 0-100% lysstyrke og videre til `duty_u16()`. Resultatet er en 'bølge' af lys der ruller hen over LED'erne i en uendelig løkke styret af `SPEED` og `TICK`.

Hvad kan det bruges til:

Viser hvordan matematiske funktioner (sinus) kan bruges til at generere bløde, kontinuerlige lysstyrke-mønstre med PWM, et typisk 'breathing light'-effekt- eksempel, der illustrerer fase-forskydning mellem flere outputs.

`pwm_hold_to_ramp.py`

Titel: Hold-knap til gradvis op-/nedjustering af lysstyrke

Hvad programmet indeholder:

Én PWM-LED (pin 5) styres af to knapper. Så længe Btn1 holdes nede, øges brightness_pct gradvist med en fast rate (RAMP_RATE % pr. sekund, skaleret med loop-intervallet TICK); Btn2 mindsker den tilsvarende. Værdien clamps til 0-100 med min()/max(). Ved ændring omregnes procenten til duty_u16 og LED'en opdateres, og den nye procent printes.

Hvad kan det bruges til:

Demonstrerer 'hold-to-repeat'-adfærd (i modsætning til enkelt-tryk-trin) til jævn, kontinuerlig regulering - typisk for fysiske dæmperknapper hvor man holder knappen inde for at rampe lyset op eller ned.

`pwm_knight_rider.py`

Titel: 'Knight Rider'-scanner-effekt

Hvad programmet indeholder:

Seks PWM-LED'er er sat i en fysisk venstre-til-højre rækkefølge i en liste. show_eye_at(index) sætter lysstyrken for hver LED ud fra dens afstand til det aktive 'øje': selve positionen lyser 100%, én LED væk lyser 40%, to LED'er væk 15%, resten er slukket (TRAIL-listen). Positionslisten genereres som en frem- og-tilbage-sekvens (0..5..0), så 'øjet' sveper frem og tilbage i en uendelig løkke med STEP_DELAY mellem hvert trin.

Hvad kan det bruges til:

Klassisk visuel effekt der viser, hvordan man kombinerer flere PWM-outputs med forskellig, afstandsafhængig lysstyrke for at skabe et 'bevægeligt' lys med haleeffekt - god øvelse i indeksering og listemanipulation kombineret med PWM.

`pwm_dimmer_presets.py`

Titel: Lysstyrke-presets med knap-cyklus og blink-toggle

Hvad programmet indeholder:

Én PWM-LED (gul) styres via to knapper: Btn1 cykler gennem en liste af faste lysstyrke-presets (25/50/75/100%), Btn2 slår blink til/fra. En tidsbaseret løkke (samme tick-mønster

som i `to_leds_independent`) tjekker knapperne med en simpel sleep-baseret debounce og styrer et separat blink-flag (blinking) med fast `BLINK_HALF_PERIOD`, uafhængigt af hvilket preset der er valgt. `current_duty()` slår altid det aktuelle preset op, så blink og preset-skift kan kombineres.

Hvad kan det bruges til:

Viser hvordan man bygger en lille 'menu'-lignende brugerflade med to knapper - én til at vælge blandt faste værdier, én til at slå en funktion til/fra - og hvordan tilstand (preset + blink) håndteres samtidigt i én løkke.

`pwm_reaction_game.py`

Titel: Reaktionstids-spil (rød/grøn lys)

Hvad programmet indeholder:

Bruger to PWM-LED'er (rød = vent, grøn = go) og én knap. `play_round()` tænder rød og venter et tilfældigt interval (via `urandom` og `ticks_ms/ticks_diff` til præcis, ikke-blokerende tidsmåling), mens den løbende tjekker for for-tidligt tryk ('Too soon!'). Når ventetiden er omme, skiftes til grøn, tidspunktet gemmes (`go_time`), og programmet venter på knaptryk. Reaktionstiden beregnes med `ticks_diff()` og printes i millisekunder.

Hvad kan det bruges til:

God øvelse i præcis tidsmåling med `ticks_ms()/ticks_diff()` (mere præcist end at tælle sleep-intervaller) samt i at kombinere tilfældig ventetid, LED-feedback og knap-baseret måling til et selvstændigt reaktionsspil.

`pwm_buzzer_tone.py`

Titel: Buzzer-tone med PWM som lydgenerator

Hvad programmet indeholder:

Bruger PWM ikke til dæmpning, men til at generere lyd: `freq()` sætter selve tonehøjden (fx 440 Hz = A4), og `duty_u16()` sat til 50% (`HALF_DUTY = 32768`) skaber en ren firkantbølge til buzzeren. Brugeren angiver tonefrekvens (100-2000 Hz) og varighed (0-5 sekunder) via `get_int()`. Funktionen `beep()` sætter frekvens og duty, venter varigheden og slukker (duty 0) igen.

Hvad kan det bruges til:

Illustrerer en helt anden anvendelse af PWM end LED-dæmpning: lydgenerering, hvor `freq()` styrer den hørbare tone, og `duty cycle` blot definerer bølgeformens symmetri (lydstyrke/klang) frem for lysstyrke.

pwm_crossfade_two_leds.py

Titel: Crossfade mellem to LED'er

Hvad programmet indeholder:

To PWM-LED'er (rød og grøn) krydsblendes kontinuerligt: brugeren angiver en ønsket crossfade-tid (0-10 sekunder) via `get_int()`, som deles op i 100 trin (STEPS) med en beregnet `step_delay`. I en uendelig løkke øges rødt fra 0-100%, mens grønt samtidig falder fra 100-0% (og omvendt på vejen tilbage), ved at sætte begges `duty_u16()` ud fra `pct_to_duty()` i hvert trin.

Hvad kan det bruges til:

Simpelt, men klassisk eksempel på synkroniseret, modsatrettet dæmpning af to PWM-kanaler, relevant for at forstå, hvordan man tidsstyrer og synkroniser.

7-segment display-programmer

seven_segment_digits.py

Titel: Genbrugsmodul til 7-segment display (grundmodul)

Hvad programmet indeholder:

Selvstændigt modul, der samler al opsætning for det ene 7-segment-display: `SEGMENT_PINS` binder hvert segment-bogstav (A-G, DP) til det rigtige GPIO-ben (pin 8-15), og `DIGITS` er en opslagstabel med hvilke segmenter der skal tændes for hvert ciffer 0-9. `clear()` slukker alle segmenter inkl. prikken, og `show_digit(number, dot=False)` validerer input, rydder displayet og tænder kun de segmenter det ønskede ciffer kræver (samt evt. DP). Et `if __name__ == "__main__"`-blok gør, at filen kan køres alene som en selvtest, der blinker 0-9 igennem, uden at det sker når modulet importeres andetsteds fra.

Hvad kan det bruges til:

Selve byggestenen for alle de øvrige 7-segment-programmer: ved at samle pin- og ciffertabel i ét modul kan hvert program bare importere `seven_segment_digits` og kalde `show_digit()/clear()` uden at gentage opsætningen - det direkte svar på opgavekravet om 'et python-modul som indeholder de 10 tal'.

seven_segment_hex.py

Titel: Udvidelse til hexadecimale cifre (0-F)

Hvad programmet indeholder:

Bygger videre på `seven_segment_digits.py` ved at importere dets `SEGMENT_PINS`, `DIGITS` og `clear()` og tilføje `HEX_LETTERS` - segmentmønstre for de seks hex-bogstaver A-F (værdi 10-15), skrevet med de samme segment-navne (A-G). `ALL_DIGITS` samler de to tabeller (`{**DIGITS, **HEX_LETTERS}`), og `show_hex_digit(value, dot=False)` fungerer som `show_digit()`, men accepterer hele intervallet 0-15. En selvtest kører 0-F igennem, hvis filen køres direkte.

Hvad kan det bruges til:

Viser hvordan et eksisterende modul kan udvides uden at ændre det - ved at importere dets data og tilføje nye opslag - så displayet kan bruges til hex-tal, fx til at vise adresser, farvekoder eller andet der naturligt skrives i hexadecimal.

`type_a_digit.py`

Titel: Vis et indtastet ciffer på 7-segment display

Hvad programmet indeholder:

Meget kort program: importerer `seven_segment_digits`, spørger gentagne gange om et ciffer (0-9) via den validerede `get_int()`, og viser det med `seg.show_digit()`. Løkken kører uendeligt, indtil Ctrl+C, hvor `seg.clear()` rydder displayet.

Hvad kan det bruges til:

Det simplest mulige eksempel på at bruge modulet fra et andet program - god 'hello world' for 7-segment-displayet og et hurtigt værktøj til at teste, at alle cifre vises korrekt efter opkobling.

`blink_digit.py`

Titel: Blinkende ciffer med valgt frekvens

Hvad programmet indeholder:

Beder brugeren om hvilket ciffer der skal vises (0-9) og en blink-frekvens (1-10 Hz) via `get_int()`, beregner `half_period` ud fra frekvensen (samme beregning som i PWM-blink-programmerne) og skifter derefter i en uendelig løkke mellem `seg.show_digit(digit)` og `seg.clear()` med `half_period`-pauser imellem. Da segmenterne er almindelige digitale outputs (ikke PWM), er der ingen lysstyrkeregulering - kun rent tændt/slukket-blink.

Hvad kan det bruges til:

Overfører blink-mønstret fra PWM-LED-programmerne til 7-segment-displayet: viser at samme `half_period`-baserede on/off-teknik virker uafhængigt af hvilken slags digitalt output (enkelt LED, flere LED'er eller et helt 7-segment-ciffer) der styres.

scroll_number.py

Titel: Rul et flercifret tal hen over displayet

Hvad programmet indeholder:

Da displayet kun kan vise ét ciffer ad gangen, håndterer dette program større tal (0-99999) ved at splitte dem op i enkeltcifre (`digits = [int(d) for d in str(number)]`) og vise dem efter hinanden med 0,6 sekunders mellemrum, efterfulgt af en kort pause med tomt display, før brugeren kan indtaste et nyt tal via `get_int()`. Kører i en uendelig løkke, indtil Ctrl+C rydder displayet.

Hvad kan det bruges til:

Løser den praktiske begrænsning ved kun ét 7-segment-ciffer: et flercifret tal (fx en score eller måling) kan stadig 'vises' ved at rulle cifrene et ad gangen - relevant hvis en opgave kræver visning af tal større end 9 på samme hardware.

countdown.py

Titel: 'Rocket launcher'-nedtælling på 7-segment display

Hvad programmet indeholder:

Beder om et starttal (0-9) og en trin-varighed (1-5 sekunder) via `get_int()`, og tæller derefter ned fra starttallet til 0 på displayet med `show_digit()` (samt print til konsollen), `sleep(step_seconds)` mellem hvert tal. Ved 0 blinker programmet 'affyring' ved at vise ciffer 8 med prikken tændt seks gange (skifter mellem `show_digit(8, dot=True)` og `clear()`), inden 'Lift off!' printes.

Hvad kan det bruges til:

Den klassiske 'raket-nedtælling'-eksamensopgave fra 7-segment-slides, nu bygget oven på det genbrugelige modul i stedet for at gentage pin-opsætningen - viser, hvor kort selve hovedprogrammet bliver, når modulet håndterer al detaljen.

cycle_demo.py

Titel: Automatisk gennemløb af alle cifre (ledningstest/demo)

Hvad programmet indeholder:

Spørger om en forsinkelse mellem cifre (0,1-2,0 sekunder) via den nye `get_float()`-variant af inputvalideringen, og løber derefter uendeligt igennem 0-9 på displayet med den valgte hastighed, indtil Ctrl+C rydder det.

Hvad kan det bruges til:

Praktisk hjælpeværktøj snarere end en 'opgave': god til hurtigt at tjekke, at alle otte segmenter (og dermed alle GPIO-forbindelser) virker korrekt efter opkobling, eller som en simpel baggrundsdemo. Introducerer også `get_float()` som variant af det velkendte `get_int()`-mønster.

stopwatch.py

Titel: Stopur med præcis tidtagning (`ticks_ms/ticks_diff`)

Hvad programmet indeholder:

Tæller forløbne hele sekunder siden start (`ticks_ms()` ved opstart) og viser encifferet (0-9, ruller rundt efter 9) på displayet. I stedet for kun at bruge `sleep()` til at holde styr på tiden - hvilket ville drive langsomt, fordi `show_digit()` selv tager lidt tid at køre - beregnes `elapsed_seconds` hver gang ud fra `ticks_diff(ticks_ms(), start)`, og displayet opdateres kun, når cifret rent faktisk ændrer sig (`digit != last_shown`), mens løkken selv polles hyppigt (`sleep(0.05)`) uden at belaste CPU'en unødigt.

Hvad kan det bruges til:

God øvelse i præcis, drift-fri tidtagning: viser hvorfor man bør basere tidsmåling på `ticks_ms()/ticks_diff()` i stedet for at akkumulere `sleep`-kald, især når selve visningen (`show_digit`) tager en vis eksekveringstid.

guess_the_number.py

Titel: 'Gæt tallet'-spil med 7-segment feedback

Hvad programmet indeholder:

Pico'en vælger et hemmeligt ciffer (0-9) med `random.randint()`, og brugeren gætter via konsollen med `get_int()`. Hvert gæt vises på displayet med `seg.show_digit(guess)`, og programmet svarer 'Higher!'/ 'Lower!' afhængigt af om gættet er for lavt eller højt. Ved korrekt gæt blinker displayet ciffer 8 med prik tændt seks gange (samme 'fejring'-mønster som i `countdown.py`), og et nyt hemmeligt tal vælges til næste runde.

Hvad kan det bruges til:

Kombinerer 7-segment-visning med spillogik og brugerinteraktion: god eksamensrelevant demonstration af at bruge det genbrugelige display-modul i et lidt større program med tilstand (det hemmelige tal) og gentagne runder.

Pulse Width Modulation (PWM) teori

PWM (Pulse Width Modulation) regulerer lysstyrke/effekt ved hurtigt at tænde og slukke et signal. Duty cycle er forholdet (i %) mellem "on"-tid og "off"-tid indenfor én periode — 50% duty cycle betyder tændt halvdelen af tiden.

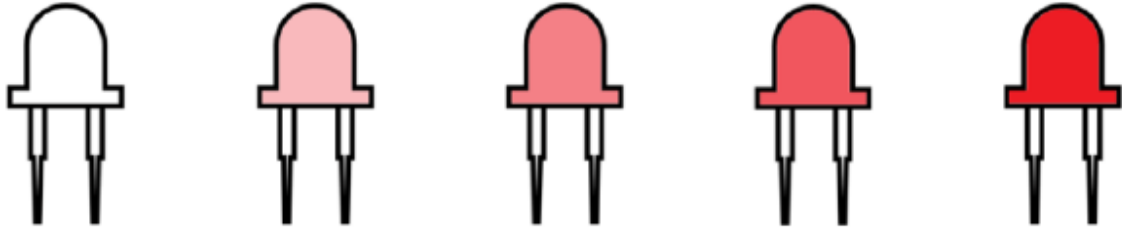


Figure 3, PWM led fra on til off

VÆR OPMÆRKSOM

PWM-carrier-frekvensen (`pwm.freq(1000)`) er den interne, hurtige switching-frekvens, der bruges til selve dæmpningen. Den er ikke det samme som en blinkefrekvens, du selv styrer med `sleep()` i en `while`-løkke.

Duty cycle (arbejdscyklus)

Arbejdscyklussen beregnes som forholdet mellem on-tiden og den samlede periode. Ved at bruge den tidligere beregnede periode beregnes arbejdsyklussen som:

PWM: Spændingsregulering

PWM-signalet giver en varierende spænding ved forskellige duty cycles. Denne metode anvendes inden for flere områder, såsom:

- **Switching-regulatorer**
- **LED-dæmpere**
- **Lydsignaler**
- **Generering af analoge signaler**
- **...og mange flere**

Spændingsregulering opnås ved at udjævne PWM-signalet. Udgangsspændingen beregnes med følgende formel:

Som det fremgår af formelen, kan udgangsspændingen direkte varieres ved at ændre **Ton**-værdien.

- Hvis **Ton** er 0, er **Vout** også 0.
- Hvis **Ton** er lig med den samlede periode **Ttotal**, er **Vout** lig med **Vin** (maksimal spænding),

Duty cycle visualiseret

Jo højere duty cycle, jo tættere ligger den gennemsnitlige ("den værdi du ser") spænding på MAX. Perioden er tiden for én hel on/off-cyklus; duty er andelen af perioden, hvor signalet er højt.

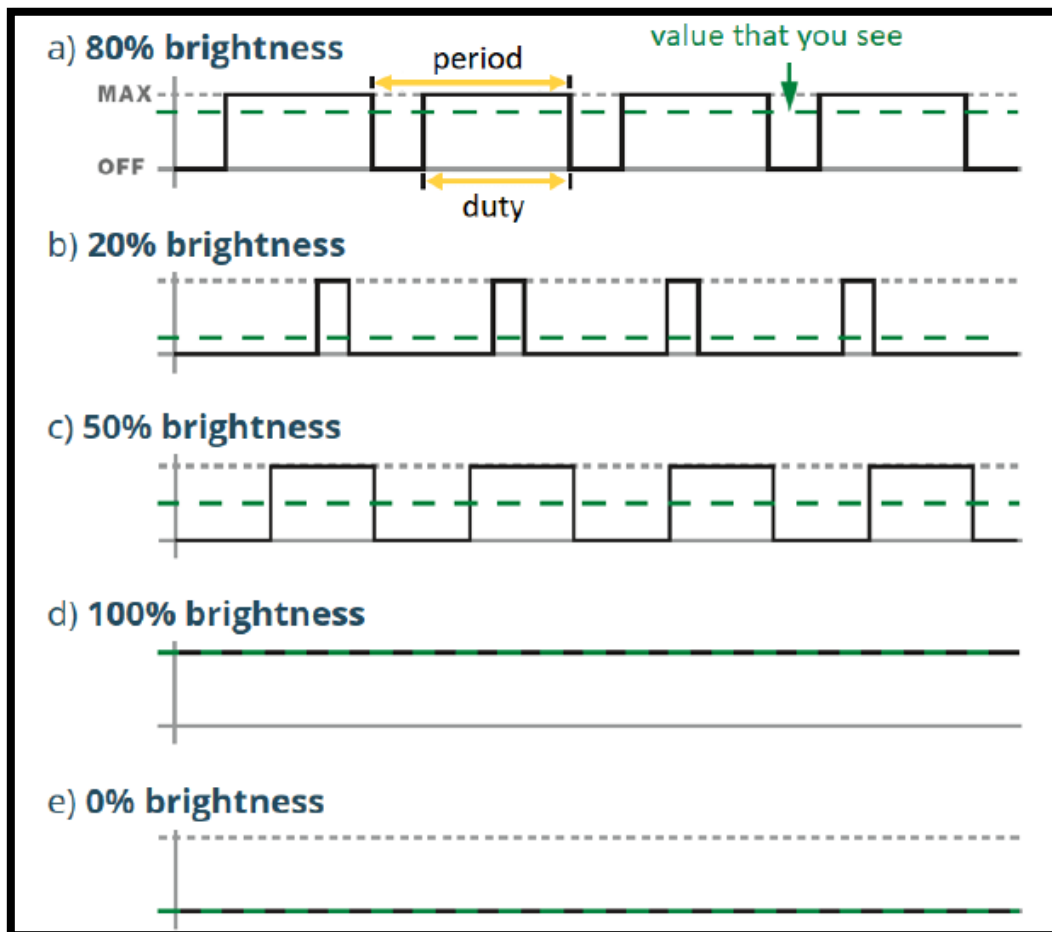


Figure 4, duty cycle graf

Øjet/lyssensoren opfatter ikke de hurtige tænd/sluk-pulser enkeltvis — det du ser, er gennemsnittet af signalet over tid. Det er præcis derfor PWM virker som "analog" lysstyrke-regulering, selvom signalet reelt kun er digitalt tændt eller slukket.

Raspberry Pi Pico — PWM-kanaler

Egenskab	Beskrivelse
Antal slices	8 uafhængige PWM-generatorer ("slices"), hver med 2 kanaler = 16 kanaler i alt
Frekvensområde	Ca. 8 Hz til 62,5 MHz (mikrocontrolleren kører internt ved 125 MHz)
Delt frekvens	De to kanaler i samme slice kører altid ved samme frekvens, men kan have hver sin duty

Eksempel: åndende LED (fade ind og ud)

Her styrer den ydre `sleep(0.02)`-løkke, hvor hurtigt lyset stiger/falder (det du kan se), mens `pwm.freq(1000)` er den usynlige, interne switching-frekvens.

```
from machine import Pin, PWM
from time import sleep
led = PWM(Pin(2))
led.freq(1000)

try:
    while True:
        for duty in range(0, 65536, 2000): # fade ind
            led.duty_u16(duty)
            sleep(0.02)

        for duty in range(65535, -1, -2000): # fade ud
            led.duty_u16(duty)
            sleep(0.02)

except KeyboardInterrupt:
    led.duty_u16(0)
    led.deinit()
```

Eksempel: lysstyrke styret af et potentiometer (ADC + PWM)

En meget almindelig opgave: brug et potentiometer til at style en LED's lysstyrke i realtid. Fordi `adc.read_u16()` og `pwm.duty_u16()` begge arbejder i samme talområde (0–65535), kan du sende ADCværdien direkte videre til PWM'en — uden at skulle regne den om.

```
from machine import Pin, PWM, ADC
from time import sleep

led = PWM(Pin(2)) # ledGL fra dit print
pot = ADC(Pin(26)) # potentiometer på ADC0

led.freq(5000)

try:
    while True:
        pot_value = pot.read_u16()
        # Slå helt fra ved meget lave værdier, så LED'en
        # ikke "gløder" svagt pga. støj tæt på 0

        if pot_value <= 420:
            led.duty_u16(0)

        else:
            led.duty_u16(pot_value)
            sleep(0.1)

except KeyboardInterrupt:
    led.duty_u16(0)
    led.deinit()
```

VÆR OPMÆRKSOM

Et potentiometer giver sjældent en helt ren 0 i sin nederste ende – der kan ligge et par hundrede "støjende" enheder, som ellers ville få LED'en til svagt at gløde, selvom den burde være helt slukket. En simpel tærskel (her 420) løser det.

Mål duty cycle med et oscilloskop

Vil du se PWM-signalet med egne øjne i stedet for at regne det ud, kan du bruge et oscilloskop med to probe'r:

- Den ene probe sættes på selve PWM-benet (fx GPIO 2).
- Den anden probe (eller probens jordklemme) sættes på GND, så du har et fælles referencepunkt.

Skoperet tegner signalet som en firkantbølge, hvor du direkte kan aflæse periodens længde (og dermed frekvensen) samt hvor stor en andel af perioden der er høj (duty cyclen), en god måde at bekræfte, at koden faktisk gør det, du tror den gør.

ADC, A/D og spændingsdeler teori

En ADC (Analog-til-Digital Converter) omdanner et analogt spændingssignal til en digital værdi, mikrocontrolleren kan regne videre på. Pico'ens ADC-indgang (GPIO 26 / ADC0) måler kun 0–3,3 V.

- Pico kan max måle på 3,3V
- Analog værdi = 0V : Digital værdi = 0
- Analog værdi = 1,65V : Digital værdi = 32767
- Analog værdi = 3,3V : Digital værdi = 65535

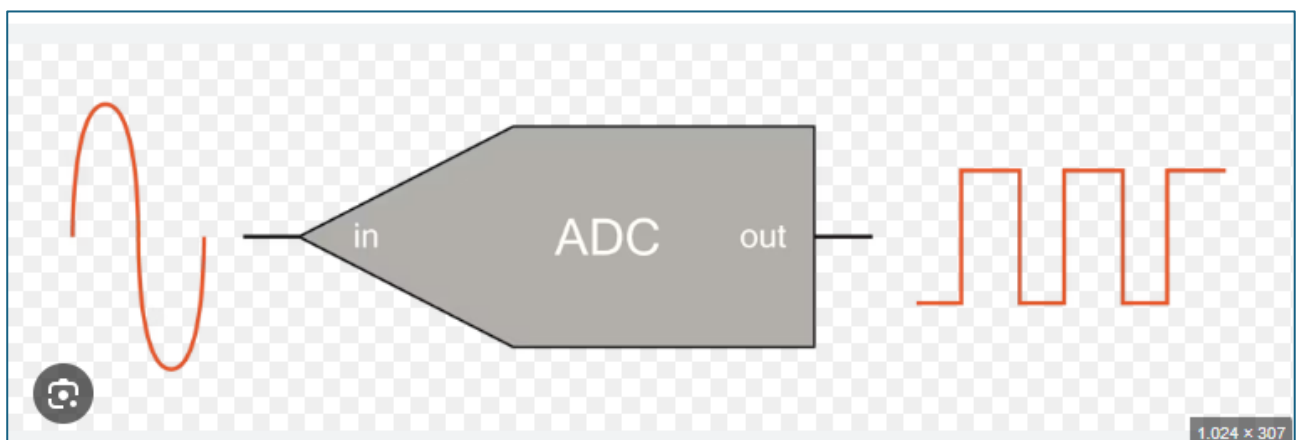


Figure 5, diagram for analog til digital konvertering (A/D)

VÆR OPMÆRKSOM HVIS DU SKAL MÅLE MERE END 3,3 V

Fx et 6,4–8,4 V robotbatteri eller 0–12 V, så skal du bruge en spændingsdeler (to modstande) FØR signalet rammer ADC indgangen, og regne den rigtige spænding ud igen i softwaren.

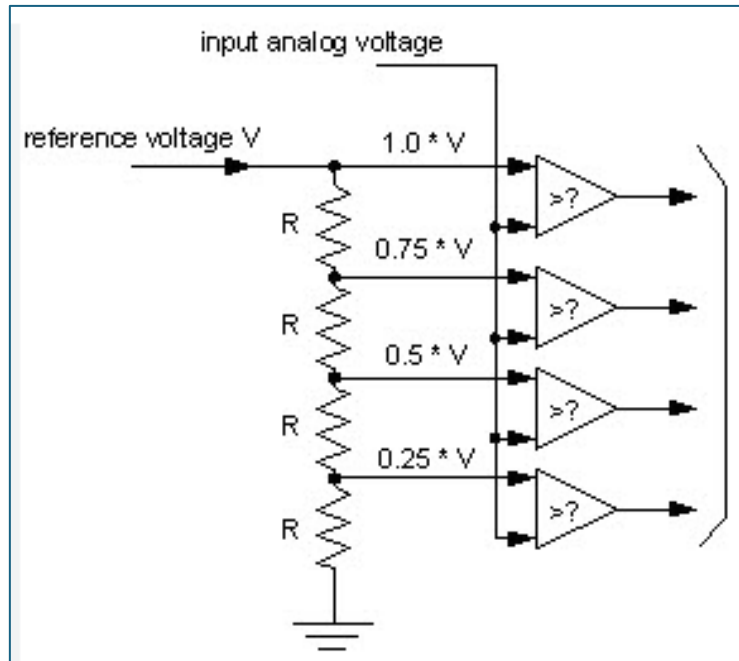


Figure 6, A/D praktisk

Sådan tænker du en spændingsdeler

Sæt to modstande R_1 og R_2 i serie fra din høje spænding ned til GND, og mål på punktet mellem dem. Formlen for hvad ADC'en reelt ser er:

```
adc_voltage = adc.read_u16() / 65535 * 3.3
real_voltage = adc_voltage / (R2 / (R1 + R2))
```

Jo større R_1 er i forhold til R_2 , jo mere "skalerer" du den høje spænding ned til noget, Pico'en kan tåle — men husk altid at gange tilbage i koden for at få den rigtige værdi at vise.

Statemachines teori

En statemachine strukturerer en proces som en række states (tegnes som cirkler), med pile der viser hvilken state man skifter til og hvorfor. I koden svarer hver cirkel til én funktion, som udfører sit arbejde og returnerer (ikke kalder) den næste state-funktion.

KILDE: Statemachine v2.pptx

Læs mere i Statemachine v2.pptx

Eksempel: hysteresese ved en termostat

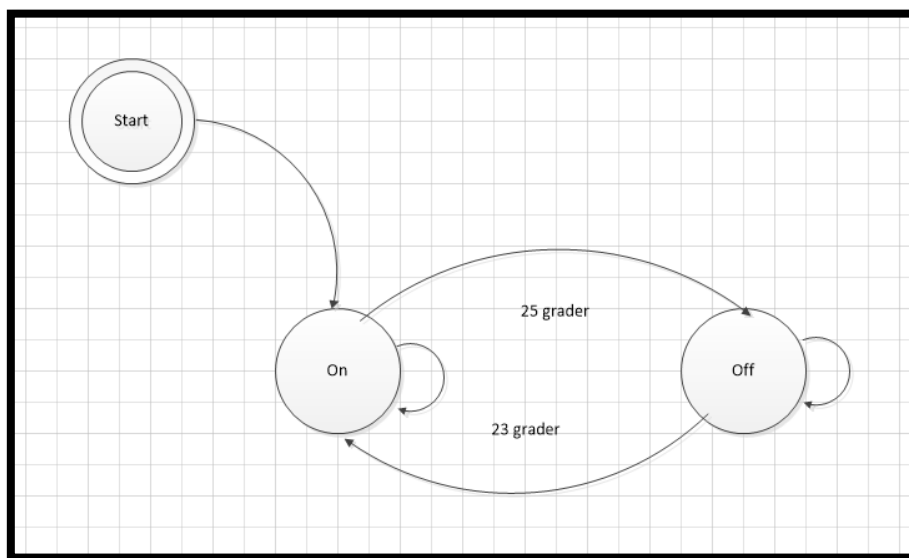


Figure 7, hysteresese sløjfe

Uden hysteresese ville en termostat "flimre" tændt/slukket lige omkring ét sætpunkt, fx 20°C. Med hysteresese bruges to grænser i stedet:

```
ON_GRÆNSE = 19 # tænd varmen, hvis temperatur falder under denne
OFF_GRÆNSE = 21 # sluk varmen, når temperatur stiger over denne

def state_off():
    if temperatur() < ON_GRÆNSE:
        return state_on
    return state_off

def state_on():
    if temperatur() > OFF_GRÆNSE:
        return state_off
```

```
return state_on
```

Fordi de to grænser er forskellige (19 og 21, ikke begge 20), skifter systemet ikke frem og tilbage for hvert lille sving omkring 20°C.

Eksempel på en statemachine:

```
from machine import Pin
from time import sleep

def get_int(prompt, low, high):
    while True:
        try:
            value = int(input(prompt))
            if low <= value <= high:
                return value
            print(f"Enter a value between {low} and {high}.")
        except ValueError:
            print("Please type a whole number.")

ledRL = Pin(0, Pin.OUT)
ledYL = Pin(1, Pin.OUT)
ledGL = Pin(2, Pin.OUT)

ledRR = Pin(3, Pin.OUT)
ledYR = Pin(4, Pin.OUT)
ledGR = Pin(5, Pin.OUT)

groen_tid = get_int("Enter a 1-20 seconds for green led: ", 1, 20)
gul_tid = get_int("Enter a 1-20 seconds for yellow led: ", 1, 10)
roed_tid = get_int("Enter a 1-20 seconds for red led: ", 1, 20)

def alle_slukket():
    ledRL.value(0)
    ledYL.value(0)
    ledGL.value(0)

    ledRR.value(0)
    ledYR.value(0)
    ledGR.value(0)

def state_groen():
    alle_slukket()
    ledGL.value(1)
    ledGR.value(1)
    print("Green... ready...")
    sleep(groen_tid)
    return state_gul
```

```

def state_gul():
    alle_slukket()
    ledYL.value(1)
    ledYR.value(1)
    print("Yellow... ready...")
    sleep(gul_tid)
    return state_roed

def state_roed():
    alle_slukket()
    ledRL.value(1)
    ledRR.value(1)
    print("Red... ready...")
    sleep(roed_tid)
    return state_groen

try:
    state = state_groen          # initial state
    while state:
        state = state()          # kør statemachine
except KeyboardInterrupt:
    alle_slukket()
    print("Stoppet, alle LED'er slukket.")

```

KILDE: Big-Beautiful-Board repository (github)

Læs mere om pin-lay out, samt flere kode eksempler på Big-Beautiful-Board repository.
<https://github.com/CN-Thrige/Big-Beautiful-Board>

Farvet konsoloutput (ANSI escape-koder)

Nyttigt hvis en opgave vil have fx "grønt lys = grøn tekst" i konsollen:

```

print('\x1b[7;32;40m' + "Grøn! Kør" + '\x1b[0m') # grøn baggrund/tekst
print('\x1b[6;30;43m' + "Gul! stop" + '\x1b[0m') # gul
print('\x1b[6;30;41m' + "Rød! stop" + '\x1b[0m') # rød

```

Formatet er \x1b[;;m og nulstilles altid med \x1b[0m bagefter — ellers "smitter" farven resten af outputtet.

Random statemachine

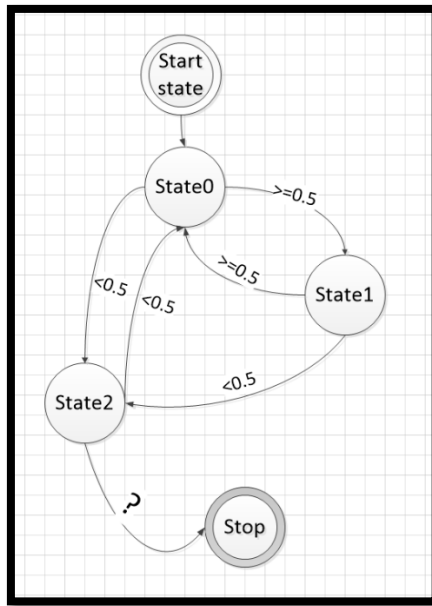


Figure 8, diagram for Random State machine

Eksempel: random_statemachine_random_getint_style.py

```

from machine import Pin
from random import random
from time import sleep

led0 = Pin(0, Pin.OUT)
led1 = Pin(1, Pin.OUT)
led2 = Pin(2, Pin.OUT)

def get_int(prompt, lo, hi):
    while True:
        try:
            val = int(input(prompt))
            if lo <= val <= hi:
                return val
            print(f"Enter a value between {lo} and {hi}.")
        except ValueError:
            print("Please type a whole number.")

step_time = get_int("Seconds per state (1-5): ", 1, 5)
stop_chance_pct = get_int("Chance to stop at State2, in % (0-100): ", 0, 100)
stop_threshold = stop_chance_pct / 100

def all_off():
    led0.off(); led1.off(); led2.off()

def state0():
    all_off(); led0.on()
    sleep(step_time)
    return state1 if random() >= 0.5 else state2

```

```

def state1():
    all_off(); led1.on()
    sleep(step_time)
    return state0 if random() >= 0.5 else state2

def state2():
    all_off(); led2.on()
    sleep(step_time)
    if random() < stop_threshold:
        return None
    return state0

try:
    state = state0
    while state:
        state = state()
        all_off()
        print("Stopped at State2.")
except KeyboardInterrupt:
    all_off()
    print("Stopped, outputs cleared.")

```

KILDE: Big-Beautiful-Board repository (github)

Læs mere om pin-lay out, samt flere kode eksempler på Big-Beautiful-Board repository.
<https://github.com/CN-Thrige/Big-Beautiful-Board>

7-segment display teori

Består af 7 uafhængige LED'er (segmenterne a–g), der tændes i forskellige kombinationer for at danne cifre 0–9, plus evt. et 8. segment (decimalpunktet). Hvert segment skal have sin egen 220 Ω modstand for ikke at trække for meget strøm.

VÆR OPMÆRKSOM

Gem cifferkombinationerne (0–9) i et separat Python-modul, du kan importere, i stedet for at skrive dem ind i hovedprogrammet hver gang – nemmere at genbruge og nemmere for censor at læse.

Eksempel 1: cifferopslag som separat modul

```
# digits.py – importeres i hovedprogrammet
# Rækkefølge: (a, b, c, d, e, f, g)

DIGITS = {
    0: (1,1,1,1,1,1,0),
    1: (0,1,1,0,0,0,0),
    2: (1,1,0,1,1,0,1),
    # ... osv. for 3-9
}
```

Eksempel 2: tænd pinene for cifret 0

```
from machine import Pin

led1 = Pin(8, Pin.OUT) # d
led2 = Pin(9, Pin.OUT) # e
led3 = Pin(10, Pin.OUT) # g
led4 = Pin(11, Pin.OUT) # f
led5 = Pin(12, Pin.OUT) # a
led6 = Pin(13, Pin.OUT) # b
led7 = Pin(14, Pin.OUT) # c
led8 = Pin(15, Pin.OUT) # dp
pins = [led5, led6, led7, led1, led2, led4, led3, led8] # a,b,c,d,e,f,g,dp
digit_0 = (1,1,1,1,1,1,0,0)

for pin, value in zip(pins, digit_0):
    pin.value(value)
```

KILDE: 7 segment.pptx

Læs mere i 7 segment.pptx

GY-53 ToF-afstandssensor teori

Time-of-Flight-sensor (baseret på VL53L0X-chippen), måleområde 0–2 meter. Kan sende data på tre måder: UART, PWM eller I²C.

I PWM-mode sender sensoren et signal med fast periode (~22 ms), og pulslængden (HIGH-tiden) er proportional med afstanden:

$\text{Afstand (mm)} = \text{Pulslængde } (\mu\text{s}) / 10$

VÆR OPMÆRKSOM

Fordel: simpel implementering, ingen adressering nødvendig. Ulempe: mindre robust mod støj, kræver præcis timing – derfor bruges `time.ticks_us()` og `ticks_diff()`, ikke almindelig `time.time()`.

Netværksværktøjer

Værktøj	Bruges til
Putty	SSH/Telnet-klient — bruges til at logge ind på en Raspberry Pi/enhed over netværket via kommandolinje..
VNC (RealVNC)	Fjernskrivebordsprogram — giver et grafisk skrivebord til en headless Raspberry P
Advanced IP Scanner	Scanner et helt IP-range (fx 10.110.0.1–10.110.0.254) og viser hvilke enheder der er på nettet med IP, MAC-adresse og producent — bruges til at finde din Pi/Pico's IP-adresse.

Bageopskrift med PWM-eksempel

Stort set alle opgaver i faget bygger på den samme grundstruktur. Lærer du denne opskrift udenad, kan du løse næsten hvad som helst — du skal blot udskifte "hvad løkken gør" fra opgave til opgave.

1. Importér det du skal bruge

MACHINE giver adgang til hardwaren (ben, PWM, ADC). time giver dig sleep() til at vente mellem handlinger.

```
from machine import Pin, PWM
from time import sleep
```

2. Opret dine pins

Brug pin-layoutet fra afsnit *Pin-layout for Big Beautiful Board*, så du ikke selv skal huske GPIO-numre.

```
ledGL = PWM (Pin (2))
```

3. Sæt PWM-frekvensen

Den interne switching-frekvens, som er typisk 1000 Hz. Sættes kun én gang.

```
led.freq (1000)
```

VÆR OPMÆRKSOM

Dette er kun relevant ved PWM.

4. Skriv selve løkken

Det er her opgaven reelt bliver løst.

```
while True:
    led.duty_u16(65535) # fuldt tændt (100%)
    sleep(0.5)
    led.duty_u16(0) # slukket (0%)
    sleep(0.5)
```

5. Det færdige program

Med try / except kan du stoppe pænt med Ctrl+C, uden at en LED eller PWM-kanal hænger tændt bagefter.

```
from machine import Pin, PWM
from time import sleep

led = PWM(Pin(2))

led.freq(1000)

try:
    while True:
        led.duty_u16(65535)
        sleep(0.5)
        led.duty_u16(0)
        sleep(0.5)

    except KeyboardInterrupt:
        led.duty_u16(0)
        led.deinit()
        print("Stoppet.")
```

Variation 1: brug en knap i stedet for tid

Erstat blot betingelsen i løkken. Husk: PULL_UP betyder tryk = 0.

```
btn_up = Pin(6, Pin.IN, Pin.PULL_UP)

if btn_up.value() == 0: # knap trykket
    led.duty_u16(65535)

else: # knap sluppet
    led.duty_u16(0)
```

Variation 2: flere "faser" (trafiklys, elevator, hysteres)

Har opgaven flere faser der afløser hinanden, erstatter du while-løkken med statemachine-skabelonen (se *Genbrugelige skabeloner*).

VÆR OPMÆRKSOM

Er du i tvivl om, hvor mange states en opgave kræver? Tegn den som cirkler og pile først, så bliver det tydeligt, hvilke funktioner du reelt skal skrive bagefter.

Genbrugelige skabeloner

Disse fem "byggeklodser" dækker næsten alle opgavetyper i faget. Genkend hvilken opgaven kræver, og tilpas skabelonen.

1. Input med validering

Brug den, hver gang opgaven beder brugeren indtaste et tal. Fx indtast en temperatur og vælg etage.

```
def get_int(prompt, low, high):  
    while True:  
        try:  
            value = int(input(prompt))  
  
            if low <= value <= high:  
                return value  
  
            print(f"> Indtast en værdi mellem {low} og {high}:")  
  
        except ValueError:  
            print("> FEJL. Indtast et helt tal.")
```

VÆR OPMÆRKSOM: HVORFOR TRY/EXCEPT HER

int("abc") vil crashe programmet med en ValueError. Ved at fange fejlen kan du bede brugeren prøve igen, i stedet for at programmet stopper.

2. Statemachine

Brug den til trafiklys, elevator, hysteres — alt der har "faser". Hver cirkel i din tegning bliver til én Pythonfunktion.

```
def state_a(): # gør noget  
    sleep(1)  
    return state_b # returnér NÆSTE funktion, kald den ikke selv  
def state_b():  
    sleep(1) return  
state_a state = state_a  
  
while state:  
    state = state()
```

Eksempel 1: Forenklet trafiklys, som statemachine

```
from machine import Pin
from time import sleep

ledRL = Pin(0, Pin.OUT)
ledYL = Pin(1, Pin.OUT)
ledGL = Pin(2, Pin.OUT)

def state_red():
    ledRL.on()
    ledYL.off()
    ledGL.off()
    sleep(2)
    return state_red_yellow

def state_red_yellow():
    ledRL.on()
    ledYL.on()
    ledGL.off()
    sleep(1)
    return state_green

def state_green():
    ledRL.off()
    ledYL.off()
    ledGL.on()
    sleep(2)
    return state_yellow

def state_yellow():
    ledRL.off()
    ledYL.on()
    ledGL.off()
    sleep(1)
    return state_red

state = state_red

try:
    while state:
        state = state()
except KeyboardInterrupt:
    ledRL.off()
    ledYL.off()
    ledGL.off()
```

VÆR OPMÆRKSOM

Læg mærke til hver funktion gør ét stykke arbejde, venter, og returnerer (uden parenteser!) den næste state. Selve while-løkken kalder funktionen for os.

3. ADC og spænding

Standard ADC kode:

```
from machine import ADC, Pin

adc = ADC(Pin(26))

voltage = adc.read_u16() / 65535 * 3.3
```

Skal du måle mere end 3,3 V (batteri, 12 V osv.), skal du bruge en spændingsdelers før Pico'ens ben og gange den rigtige spænding tilbage i koden:

```
real_voltage = adc_voltage / (R2 / (R1 + R2))
```

4. PWM

```
pwm = PWM(Pin(0))
pwm.freq(1000) # fast intern switching-frekvens
pwm.duty_u16(int(pct/100*65535)) # 0-100% lysstyrke
```

5. Knap med PULL_UP

```
button = Pin(6, Pin.IN, Pin.PULL_UP)
if button.value() == 0: # tryk = 0, ikke 1
    ...
```

Eksamen, points og svar

Dette program gav 20 points under først eksamen.

```
from machine import Pin
from time import sleep

button1 = Pin(6, Pin.IN, Pin.PULL_UP)

p1 = Pin(0, Pin.OUT) #red
p0 = Pin(1, Pin.OUT) #yellow

while True:
    if button1.value() == 0:
        p0.on()
        sleep(0.5)
        print("Yellow")
        p0.off()

        p1.on()
        sleep(0.5)
        print("Red")
        p1.off()
```

Den næste del af opgaven var PWM og til eksamen gav det kun 15 points (altså halvdelen, og det fulde antal points er 30 for anden halvdel).

```
#this gave 15 points at the exam!
from machine import Pin, PWM
from time import sleep

#this program gave 15 points doing the exam

#button1 = Pin(6, Pin.IN, Pin.PULL_UP)

led = Pin(0, Pin.OUT)

p1 = machine.Pin(0) #red
led_pwm1 = PWM(p1)

p0 = machine.Pin(1) #yellow
led_pwm2 = PWM(p0)

duty_step = int(input("type a brightness of PMW between 0 to 100%: "))
#duty_step = 129

# Set PWM frequency
freqlim = 20
frequency = int(input("type a frequency between 0 to 20Hz: "))
led_pwm1.freq(frequency)
led_pwm2.freq(frequency)

try:
    while True:

        for duty_cycle in range(0, 65536, duty_step): #65536
            led_pwm1.duty_u16(duty_cycle)
            sleep(0.005)

            led_pwm2.duty_u16(duty_cycle)
            sleep(0.005)

        for duty_cycle in range(65536, 0, -duty_step):
            led_pwm1.duty_u16(duty_cycle)
            sleep(0.005)

            led_pwm2.duty_u16(duty_cycle)
            sleep(0.005)

except KeyboardInterrupt:
    print("Keyboard interrupt")
    led_pwm1.duty_u16(0)
    led_pwm2.duty_u16(0)

    print(led_pwm1)
    print(led_pwm2)
    led_pwm1.deinit()
    led_pwm2.deinit()
```

Med hjælp fra AI har jeg fået ret 15 point version så den lever op til kravene til at få 30 points.

```
from machine import Pin, PWM
from time import sleep

led_pwm1 = PWM(Pin(0)) # red
led_pwm2 = PWM(Pin(1)) # yellow

PWM_CARRIER = 1000 # fixed internal switching freq, used only for dimming
led_pwm1.freq(PWM_CARRIER)
led_pwm2.freq(PWM_CARRIER)

def get_int(prompt, low, high):
    while True:
        try:
            value = int(input(prompt))

            if low <= value <= high:
                return value

            print(f"> Enter a value between {low} and {high}: \n")

        except ValueError:
            print("> ERROR. Please type a whole number.\n")

frequency = get_int("> Enter a frequency between 0 to 20Hz: ", 0, 20)
brightness_pct = get_int("> Enter a brightness between 0-100%: ", 0, 100)

duty = int(brightness_pct / 100 * 65535)

def set_leds(on):
    d = duty if on else 0
    led_pwm1.duty_u16(d)
    led_pwm2.duty_u16(d)

try:
    if frequency == 0:
        set_leds(True) # solid, at whatever brightness was chosen (0% = off)
        while True:
            sleep(1)
    else:
        half_period = (1 / frequency) / 2
        while True:
            set_leds(True)
            sleep(half_period)
            set_leds(False)
            sleep(half_period)
except KeyboardInterrupt:
    led_pwm1.duty_u16(0)
    led_pwm2.duty_u16(0)
    led_pwm1.deinit()
    led_pwm2.deinit()
    print("Stopped, PWM deinitialized.")
```


KILDE: Big-Beautiful-Board repository (github)

Læs mere om pin-lay out, samt flere kode eksempler på Big-Beautiful-Board repository.

<https://github.com/CN-Thrige/Big-Beautiful-Board>

Ohms lov teori, formler og regneeksempler

Gennemgang af Ohms lov, med formler til hovedregning og lommeregner, samt gennemregnede eksempler.

Ohms lov formler

Modstand i Serieforbindelse

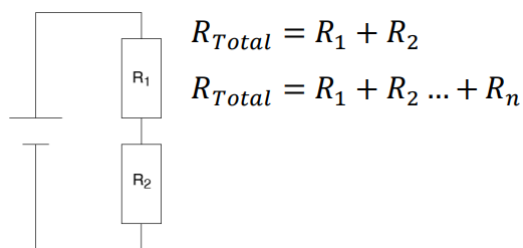


Figure 9, modstand i Serieforbindelse

Modstand i Parallelforbindelse

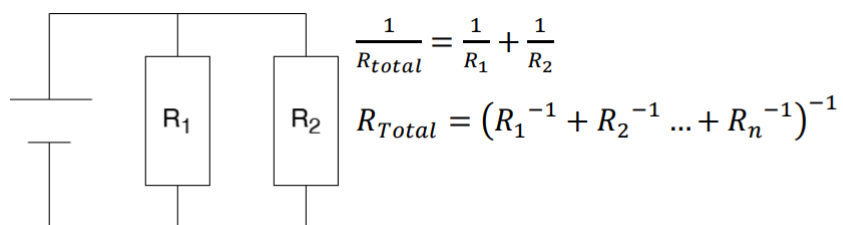
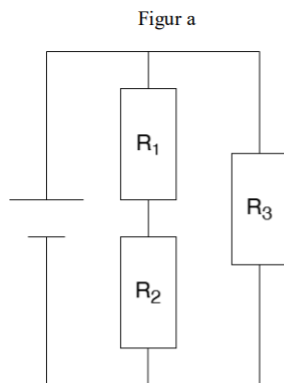


Figure 10, modstand i Parallelforbindelse

Modstand i Serie- og Parallelforbindelse

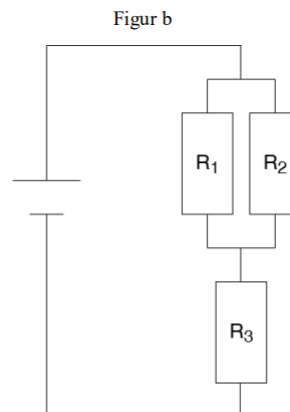


$$R_{1-2} = R_1 + R_2$$

$$R_{Total} = (R_{1-2}^{-1} + R_3^{-1})^{-1}$$

$$R_{Total} = ((R_1 + R_2)^{-1} + R_3^{-1})^{-1}$$

$$R_{Total} = R_1 + R_2 // R_3$$



$$R_{1//2} = (R_1^{-1} + R_2^{-1})^{-1}$$

$$R_{Total} = R_{1//2} + R_3$$

$$R_{Total} = (R_1^{-1} + R_2^{-1})^{-1} + R_3$$

Metode: Når man skal udregne den totale modstand i et kredsløb med både serie- og parallelforbindelser, er det vigtigt at man udregner dem i trin.

I **figur a**, skal man starte med at udregne serieforbindelsen, da den er inde i parallelforbindelsen.

I **figur b**, skal man starte med at udregne parallelforbindelsen, da den er en del af serieforbindelsen.

Figure 11, modstand i Serie- og Parallelforbindelse

Ohms lovs trekanter



To find the Voltage, (V)

$$[V = I \times R] \quad V \text{ (volts)} = I \text{ (amps)} \times R \text{ (}\Omega\text{)}$$

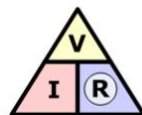
$$\textcircled{V} = I \times R$$



To find the Current, (I)

$$[I = V \div R] \quad I \text{ (amps)} = V \text{ (volts)} \div R \text{ (}\Omega\text{)}$$

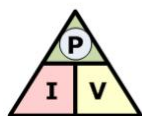
$$\textcircled{I} = \frac{V}{R}$$



To find the Resistance, (R)

$$[R = V \div I] \quad R \text{ (}\Omega\text{)} = V \text{ (volts)} \div I \text{ (amps)}$$

$$\textcircled{R} = \frac{V}{I}$$



To find the Electrical Power (P)

$$[P = V \times I] \quad P \text{ (watts)} = V \text{ (volts)} \times I \text{ (amps)}$$

$$\textcircled{P} = I \times V$$

Figure 12, ohms love trekanter

Oversigt over ohms love

Ohms Lov formler				
Kendte Værdier	Modstand (R)	Strøm (I)	Spænding (U)	Effekt (P)
Strøm & Modstand	---	---	$U = I * R$	$P = I^2 * R$
Volt & Strøm	$R = \frac{U}{I}$	---	---	$P = U * I$
Effekt & Strøm	$R = \frac{P}{I^2}$	---	$U = \frac{P}{I}$	---
Volt & Modstand	---	$I = \frac{U}{R}$	---	$P = \frac{U^2}{R}$
Effekt & Modstand	---	$I = \sqrt{\frac{P}{R}}$	$U = \sqrt{P * R}$	---
Volt & Effekt	$R = \frac{U^2}{P}$	$I = \frac{P}{U}$	---	---

Figure 13, oversigt over ohms lov figur

Præfiks

Præfiks	Navn	10potens	Tal
Z	zetta-	10^{21}	1.000.000.000.000.000.000.000
E	exa-	10^{18}	1.000.000.000.000.000.000
P	peta-	10^{15}	1.000.000.000.000.000
T	tera-	10^{12}	1.000.000.000.000
G	giga-	10^9	1.000.000.000
M	mega-	10^6	1.000.000
k	kilo-	10^3	1.000
h	hecto-	10^2	100
D	deca-	10^1	10
		10^0	1
d	deci-	10^{-1}	0,1 (1/10)
c	centi-	10^{-2}	0,01 (1/100)
m	milli-	10^{-3}	0,001 (1/1000)
μ	micro-	10^{-6}	0,000.001 (1/1.000.000)
n	nano-	10^{-9}	0,000.000.001 (1/1.000.000.000)
p	pico-	10^{-12}	0,000.000.000.001
f	femto-	10^{-15}	0,000.000.000.000.001
a	atto-	10^{-18}	0,000.000.000.000.000.001
z	zepto-	10^{-21}	0,000.000.000.000.000.000.001

Figure 14, præfiks

Enheder til Ohms lov

Symbol	Størrelse	Enhed
U	Spænding	Volt (V)
I	Strøm	Ampere (A)
R	Modstand	Ohm (Ω)

VÆR OPMÆRKSOM: PAS PÅ ENHEDERNE

Formlen kræver hele enheder. altså Ampere, ikke milliampere. Står strømmen i mA, skal du dividere med 1000 først, fx 50 mA = 0,05 A. Samme med modstand i k Ω : 2 k Ω = 2000 Ω .

Hvad siger Ohms lov?

Ohms lov beskriver sammenhængen mellem **spænding**, **modstand** og **strøm** i et elektrisk kredsløb. Jo større modstand, jo mindre strøm løber der ved samme spænding og omvendt.

Tænk på det som vand i et rør: spænding er trykket, der presser vandet igennem, modstand er hvor snævert røret er, og strøm er hvor meget vand der rent faktisk løber igennem pr. sekund. Et smallere rør (større modstand) giver mindre gennemstrømning (mindre strøm) ved samme tryk (spænding).

Ohms lov Formel

$$U = R \times I$$

Da det er en ligning, kan du altid omskrive den, hvis det er strømmen eller modstanden du mangler, i stedet for spændingen

Find spændingen: $U = I \times R$

Find strømmen: $I = U \div R$

Find modstanden: $R = U \div I$

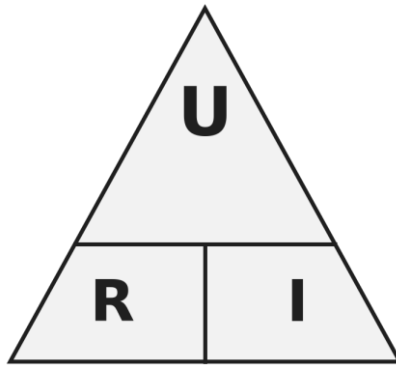


Figure 15, huskefigur: Dæk det, du søger, til med fingeren. De to andre bogstaver viser, om du skal gange eller dividere.

VÆR OPMÆRKSOM: Sådan bruger du trekanten

Dæk U til $\rightarrow$ tilbage står R og I ved siden af hinanden $\rightarrow$ gang dem: $U = R \times I$. Dæk R til $\rightarrow$ U står over I $\rightarrow$ divider: $R = U / I$. Samme princip for I.

Sådan taster du det på lommeregneren

Lommeregneren regner det samme som formlen, du skal bare taste den i rækkefølge fra venstre mod højre:

- Gange (finde U): tast tallet for R, tryk $\times$, tast tallet for I, tryk $=$.
- Dividere (finde I eller R): tast tallet der står øverst i brøken, tryk $\div$, tast tallet der står nederst, tryk $=$.

Eksempler på ohms lov

Eksempel 1: find spændingen (U)

En modstand på $R = 100 \Omega$ har en strøm på $I = 0,5 A$ igennem sig. Hvad er spændingen over modstanden (Find spændingen U)?

Formel:

$$U = R \times I$$

Indsæt tallene:

$$U = 100 \times 0,5$$

Regn ud:

$$U = 50 \text{ V}$$

- På lommeregner: $100 \times 0,5 = 50 \cdot 10^0$ (tjek figur 6 præfiks)

Eksempel 2: find strømmen (I)

Opgave: Der er 12 V over en modstand på $R = 240 \Omega$. Find strømmen I.

Formel:

$$I = U / R$$

Indsæt tallene:

$$I = 12 / 240$$

Regn ud:

$$I = 0,05 \text{ A} = 50 \text{ mA}$$

På lommeregner: $12 \div 240 = 50\text{mA}$

Eksempel 3: find modstanden (R)

Opgave: En LED har 9 V over sig og trækker en strøm på 3 mA. Find modstanden R.

Omregn først mA til A:

$$3 \text{ mA} = 0,003 \text{ A}$$

$$R = U / I$$

Indsæt tallene:

$$R = 9 / 0,003$$

Regn ud:

$$R = 3000 \Omega = 3 \text{ k}\Omega$$

På lommeregner: $9 \div 0.003 = 3 \text{ k}\Omega$

VÆR OPMÆRKSOM:

Glemmer man at omregne mA til A (eller k Ω til Ω) før man regner, bliver svaret 1000 gange forkert. Tjek altid enhederne, før du taster på lommeregneren.

Eksempel 4: Find strømmen (I) 2

Opgave: Et batteri på 12 V er koblet til en modstand på 6 Ω . Hvor stor en strøm løber der i kredsløbet?

Formel:

$$I = U / R$$

Indsæt tallene:

$$I = 12 / 6$$

Regn ud:

$$12/6 = 2 \cdot 10^0 = 2 \text{ A}$$

Eksempler på modstand i serieforbindelse, parallelforbindelse og serie- og parallelforbindelse

Serieforbindelse

Når to eller flere modstande sidder efter hinanden i samme ledningsbane, kaldes det en serieforbindelse. Strømmen har kun én vej at løbe, så den samme strøm løber igennem alle modstandene.

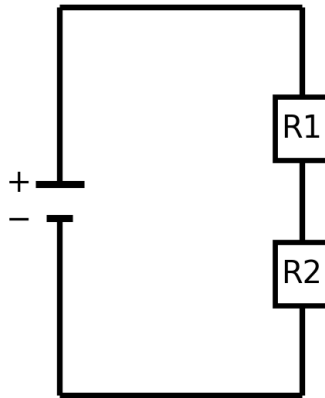


Figure 16, modstand i serieforbindelse

Serieforbindelse formel:

$$R_{\text{Total}} = R_1 + R_2$$

$$R_{\text{Total}} = R_1 + R_2 \dots + R_n$$

VÆR OPMÆRKSOM

Ved en serieforbindelse lægger du bare modstandene sammen. Den samlede modstand bliver altid større end den største enkeltmodstand.

Eksempel: 2 modstande i serie

Opgave: $R_1 = 10 \, \Omega$ og $R_2 = 20 \, \Omega$ er koblet i serie. Find den samlede modstand R_{Total} .

Formel

$$R_{\text{Total}} = R_1 + R_2$$

Indsæt tallene:

$$R_{\text{Total}} = 10 + 20$$

Regn ud:

$$R_{\text{Total}} = 30 \, \Omega$$

På lommeregner: $10 + 20 = 30 \cdot 10^0$

Modstand i parallelforbindelse

Når to eller flere modstande sidder side om side, så strømmen kan vælge mellem flere veje, kaldes det en parallelforbindelse. Spændingen er den samme over hver modstand, men strømmen deler sig mellem grenene.

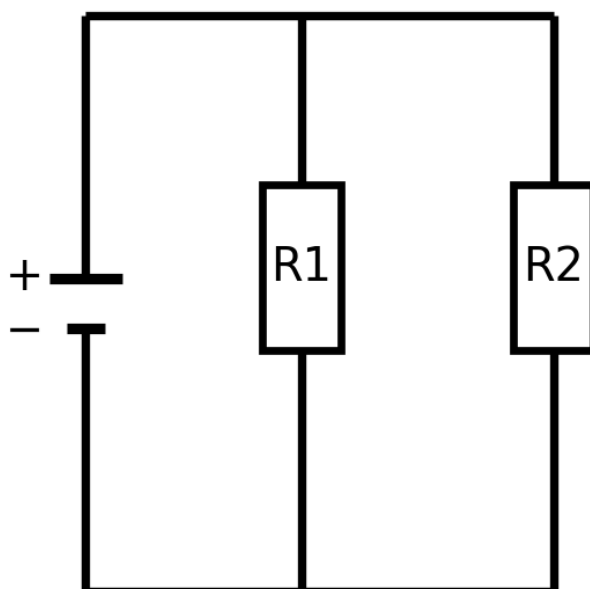


Figure 17, modstand i parallelforbindelse

Formel for parallelforbindelse

$$\frac{1}{R_{\text{Total}}} = \frac{1}{R_1} + \frac{1}{R_2}$$
$$R_{\text{Total}} = (R_1^{-1} + R_2^{-1} \dots + R_n^{-1})^{-1}$$

VÆR OPMÆRKSOM

Når der kun er to modstande i parallel, kan du bruge en meget lettere formel: $R_{\text{Total}} = (R_1 \times R_2) \div (R_1 + R_2)$. Den er nemmere at taste på lommeregner end den generelle formel

Eksempel — to modstande i parallel

Opgave: $R_1 = 10 \, \Omega$ og $R_2 = 20 \, \Omega$ er koblet parallelt. Find den samlede modstand R_{Total} .

Formel:

$$R_{\text{Total}} = (R_1 \times R_2) \div (R_1 + R_2)$$

Indsæt tallene:

$$R_{\text{Total}} = (10 \times 20) \div (10 + 20)$$

Regn ud:

$$R_{\text{Total}} \approx 6,67 \, \Omega$$

På lommeregner: $(10 \times 20) \div (10 + 20) = 6.67 \times 10^0$

VÆR OPMÆRKSOM:

Den samlede modstand ved en parallelforbindelse bliver altid mindre end den mindste enkeltmodstand.

Modstand i serie- og parallelforbindelse

Mange kredsløb er en blanding af serie- og parallelforbindelser. Metoden er at udregne dem i trin, ét stykke ad gangen — start altid med den forbindelse, der sidder "inde i" den anden.

Figur a: serieforbindelse inde i en parallelforbindelse

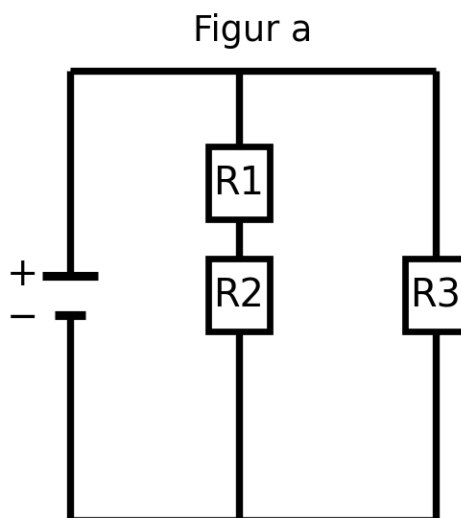


Figure 18, figur a, Serieforbindelse inde i en parallelforbindelse

R_1 og R_2 sidder i serie med hinanden, og denne kombination sidder parallelt med R_3 . Her skal du starte med serieforbindelsen ($R_1 + R_2$), fordi den sidder inde i parallelforbindelsen.

Formel:

$$R_1 + R_2 = R_1 + R_2$$

$$R_{\text{Total}} = (R_1 + R_2 \times R_3) \div (R_1 + R_2 + R_3)$$

Eksempel: figur a med tal

Opgave: $R_1 = 10 \Omega$ og $R_2 = 20 \Omega$ sidder i serie, og denne kombination sidder parallelt med $R_3 = 15 \Omega$. Find R_{Total} .

Trin 1: udregn seriedelen først (den sidder inde i parallelforbindelsen):

$$R_{1+2} = R_1 + R_2 = 10 + 20 = 30 \Omega$$

Trin 2: brug parallel-genvejen på R_{1+2} og R_3 :

$$R_{\text{Total}} = (R_{1+2} \times R_3) \div (R_{1+2} + R_3)$$

$$R_{\text{Total}} = (30 \times 15) \div (30 + 15)$$

$$R_{\text{Total}} = 10 \Omega$$

På lommeregner: $(30 \times 15) \div (30 + 15) = 10 \times 10^0$

Figur b: parallelforbindelse inde i en serieforbindelse

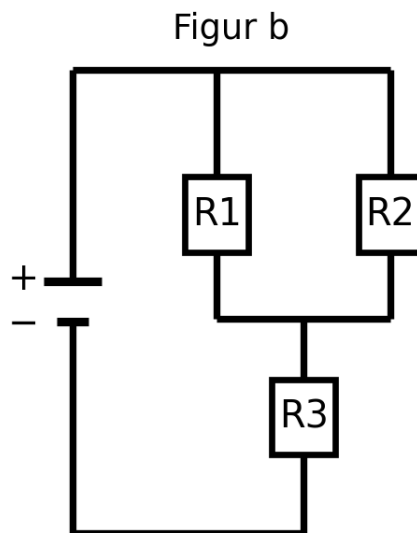


Figure 19, figur b, parallelforbindelse inde i en serieforbindelse

R_1 og R_2 sidder parallelt med hinanden, og denne kombination sidder i serie med R_3 . Her skal du starte med parallelforbindelsen (R_1 og R_2), fordi den er en del af serieforbindelsen.

$$R_1 // R_2 = (R_1 \times R_2) \div (R_1 + R_2)$$
$$R_{\text{Total}} = R_1 // R_2 + R_3$$

Sådan taster du Ohms lov ind på lommeregneren

Ohms lov og modstandsformlerne kræver kun gange ($\times$), division ($\div$) og plus ($+$), så du behøver ikke en avanceret lommeregner. Et par gode råd:

- Brug punktum eller komma til decimaltal, alt efter hvilken lommeregner du bruger (fx 0,5 eller 0.5).
- Tast tallene i samme rækkefølge, som de står i formlen: først tallet, så regnetegnet, så det andet tal, og til sidst "=".
- Ved brøker (fx parallelformlen): tast altid det, der står i parentes, færdigt først, før du dividerer med den anden parentes.
- Dobbelttjek altid enheden på dit svar: Volt (V) til spænding, Ampere (A) til strøm, og Ohm (Ω) til modstand.

Udregningseksempler fra undervisning

2 modstande i serie, hvor 1 er sat i parallel

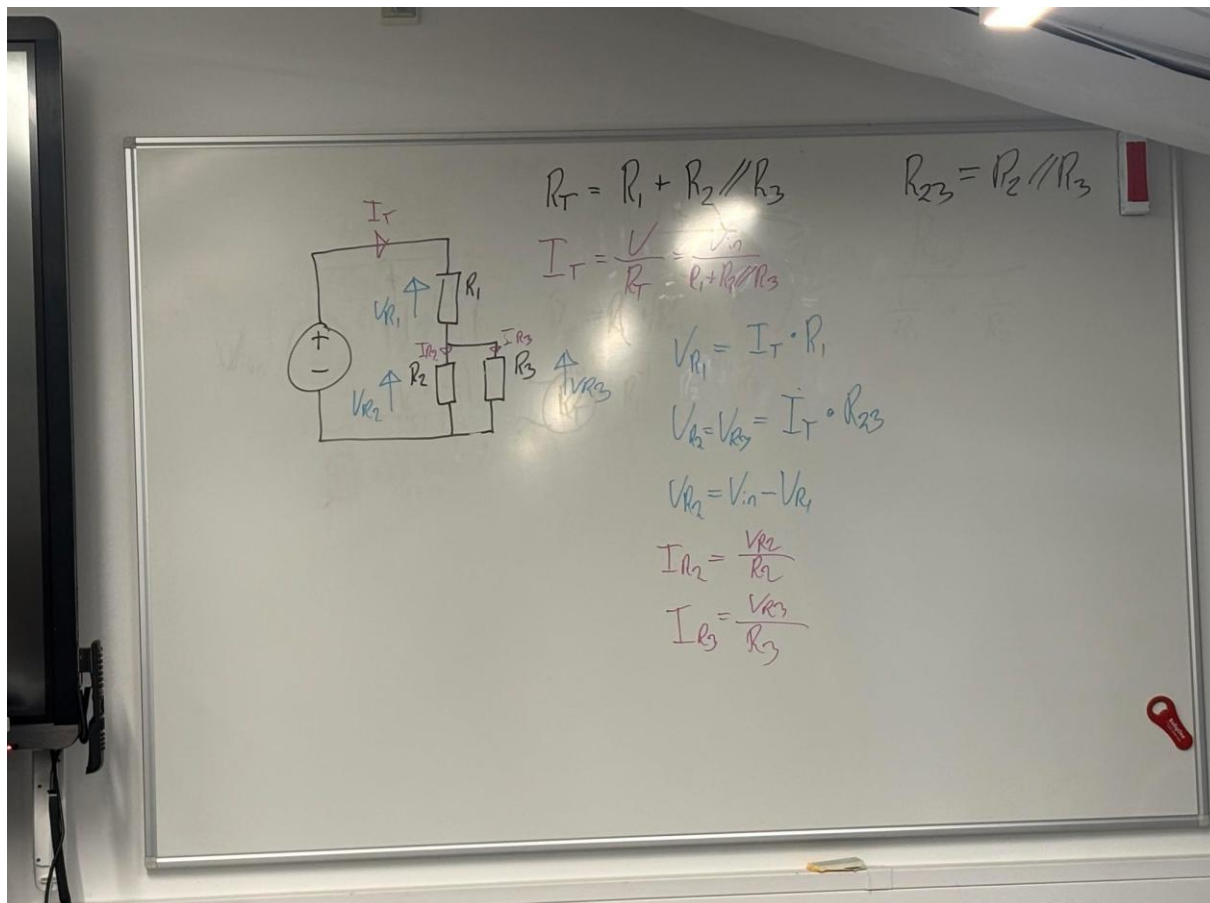


figure 20, 2 modstande i serie, hvor 1 er sat i parallel

Kredsløbsbeskrivelse

Kredsløbet består af en strømkilde (V_{in}), som er forbundet i serie med modstanden R_1 . Derefter deler kredsløbet sig op i en parallelforbindelse med to modstande, R_2 og R_3 .

VÆR OPMÆRKSOM:

Tegnet " $||$ " betyder "i parallel med".

Trin 1: Total modstand (R_{Total})

Du skal først beregne erstatningsmodstanden for de to parallelle modstande R_2 og R_3 (det gøres med formelen ovenfor). Dette resultat lægger du bagefter direkte sammen med værdien af R_1 .

$$R_{\text{Total}} = R_1 + (R_2 \parallel R_3)$$
$$R_2 \parallel R_3 = (R_2 \times R_3) / (R_2 + R_3)$$

Trin 2: Total strømstyrke (I_T)

Ohms lov ($I = V / R$) bruges til at finde ud af, hvor meget strøm der løber ud af strømkilden: du tager indgangsspændingen (V_{in}) og dividerer med den totale modstand (R_{total}) fra Trin 1.

$$I_T = V_{\text{in}} / R_{\text{total}} = V_{\text{in}} / (R_1 + R_2 \parallel R_3)$$

Trin 3: Spændingsfald over R_1 (V_1)

Hele den samlede strøm (I_T) løber igennem modstanden R_1 , før strømmen deler sig. Du ganger den totale strømstyrke med modstanden R_1 for at finde ud af, hvor mange volt der "bruges" her.

$$V_1 = I_T \times R_1$$

Trin 4: Spænding over parallelforbindelsen (V_2 og V_3)

Fordi R_2 og R_3 sidder parallelt over for hinanden, har de altid nøjagtig det samme spændingsfald.

$$\text{Metode 1: } V_2 = V_3 = I_{23} \times R_{23}$$

$$\text{Metode 2 (nemmere): } V_{23} = V_{\text{in}} - V_1$$

R_{23} er den samlede modstand for parallelforbindelsen ($R_2 \parallel R_3$). Spændingen over parallelforbindelsen er blot den spænding, der er tilbage, når du trækker spændingsfaldet i R_1 fra batteriets oprindelige spænding.

Trin 5: Strømstyrke i de enkelte grene (I_2 og I_3)

Når den samlede strøm når frem til krydset, deler den sig ud i de to parallelle veje alt efter modstandenes størrelse. Du bruger Ohms lov på hver enkelt modstand: du tager den fælles spænding for parallelforbindelsen og dividerer med værdien af den specifikke modstand.

$$I_2 = V_{23} / R_2$$

$$I_3 = V_{23} / R_3$$

2 serieforbindelser sat i parallel med hinanden

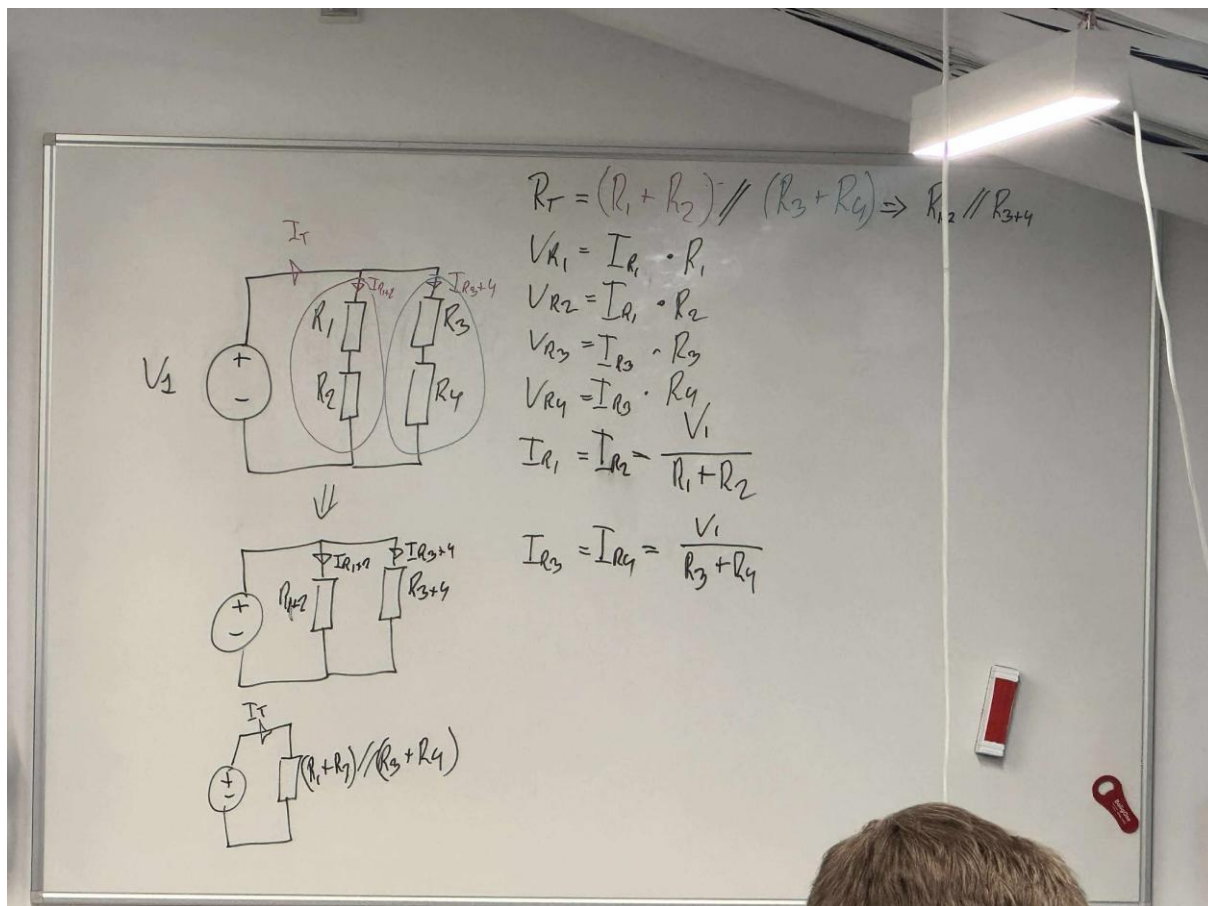


figure 21, 2 serieforbindelser sat i parallel med hinanden

Kredsløbet består af to serieforbundne grenpar (R_1+R_2 og R_3+R_4) placeret i parallel med hinanden. Nøglen i denne metode er trinvis forenkling: du lægger først modstandene sammen i hver sin gren (serie), og beregner derefter den samlede modstand for de to grene (parallel).

Den samlede modstand (R_{total})

$$R_{\text{total}} = (R_1 + R_2) \parallel (R_3 + R_4) = R_{1+2} \parallel R_{3+4}$$

Serieforbindelserne først

Da R_1 og R_2 sidder i serie i den første gren, lægges de sammen til én erstatningsmodstand:

$$R_{1+2} = R_1 + R_2$$

Det samme gælder den anden gren med R_3 og R_4 :

$$R_{3+4} = R_3 + R_4$$

Parallelforbindelsen til sidst

Symbolet || betyder parallelforbindelse. Kredsløbet er nu skåret ned til to parallelle modstande, R_{1+2} og R_{3+4} .

$$R_{\text{total}} = (R_{1+2} \times R_{3+4}) / (R_{1+2} + R_{3+4})$$

Strømmen i grenene (I)

Da begge grene er forbundet direkte over spændingskilden V_{in} , er spændingen over begge grene lig med V_{in} .

Første gren (R_1 og R_2)

Da komponenterne sidder i serie, løber den samme strøm gennem både R_1 og R_2 :

$$I_1 = I_{1+2} = V_{\text{in}} / R_{1+2}$$

Anden gren (R_3 og R_4)

På samme måde løber den samme strøm igennem R_3 og R_4 :

$$I_2 = I_{3+4} = V_{\text{in}} / R_{3+4}$$

Eksempel med fiktive tal

Formel:

$$V_{\text{in}} = 24 \text{ V} \quad - \quad \text{Gren 1: } R_1 = 10 \, \Omega, R_2 = 20 \, \Omega \quad - \quad \text{Gren 2: } R_3 = 15 \, \Omega, R_4 = 45 \, \Omega$$

Trin 1: Beregn erstatningsmodstanden for hver gren (serie)

$$\text{Gren 1: } R_{1+2} = R_1 + R_2 = 10 \, \Omega + 20 \, \Omega = 30 \, \Omega$$

$$\text{Gren 2: } R_{3+4} = R_3 + R_4 = 15 \, \Omega + 45 \, \Omega = 60 \, \Omega$$

Sådan måler du modstand med multimeter uden strøm

En trin-for-trin guide til at måle enkeltmodstande, serieforbindelser og parallelforbindelser med billeder, formler og udregningseksempler.

1 Udstyr og forberedelse

- Multimeter (digitalt) med to målespidser: én sort og én rød
- Modstande (fx dine 218 k Ω , 329 k Ω og 328 k Ω)

2 Sådan stiller du multimeteret op

Sæt den sorte ledning i porten mærket COM, og den røde ledning i porten mærket Ω (eller Ω/V , samme port bruges ofte til spænding og modstand). Drej derefter vælgerknappen hen på Ω -symbolet.

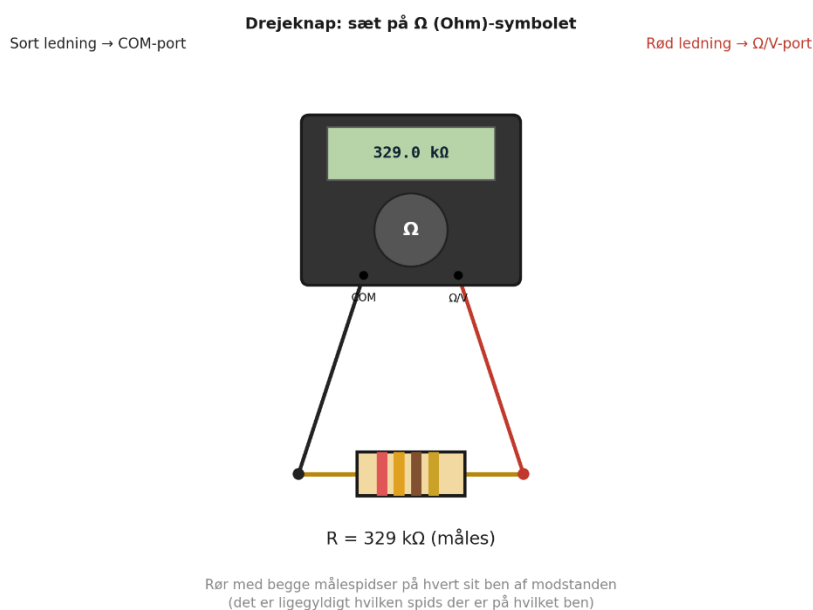


Figure 22: opsætning til måling af en enkelt modstand: sort i COM, rød i Ω/V , drejeknap på Ω .

Sådan aflæser og udfører du selve målingen

- Rør med de to målespidser på hver sin ende (hvert 'ben') af modstanden
- Det er ligegyldigt hvilken spids der rører hvilken ende — modstand har ingen retning

- Hold spidserne stille, til tallet på skærmen stabiliserer sig
- Aflæs værdien og noter enheden (Ω , $k\Omega$ eller $M\Omega$) — multimeteret vælger ofte enhed automatisk

VÆR OPMÆRKSOM:

Tip: Har modstanden farvekoder, kan du sammenligne den målte værdi med den beregnede værdi ud fra farvekoderne for at tjekke, om den ligger inden for tolerancen (typisk $\pm 5\%$).

3 Måling af serieforbindelse

Når modstande sidder i serie, er der kun én vej for strømmen, modstandene sidder efter hinanden i en kæde.

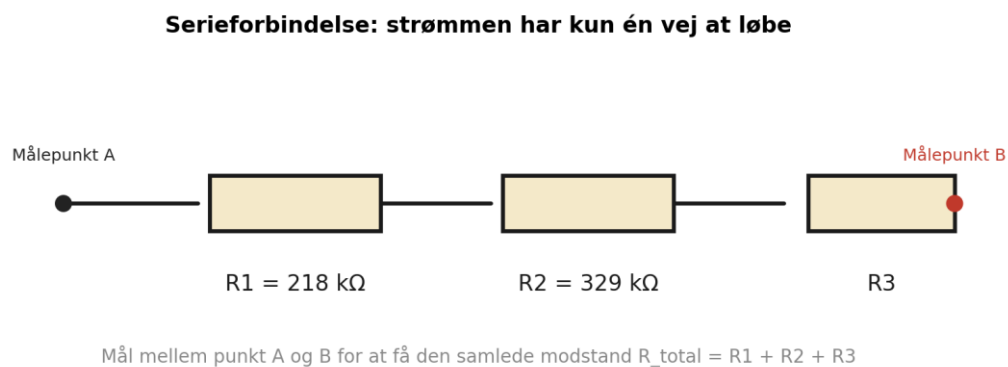


Figure 23, tre modstande i serie. Mål mellem det yderste punkt i hver ende (A og B).

På et breadboard(fumlebræt) ser det sådan her ud:

Sådan ser det ud på et breadboard: modstandene sidder på række, hver ende for sig

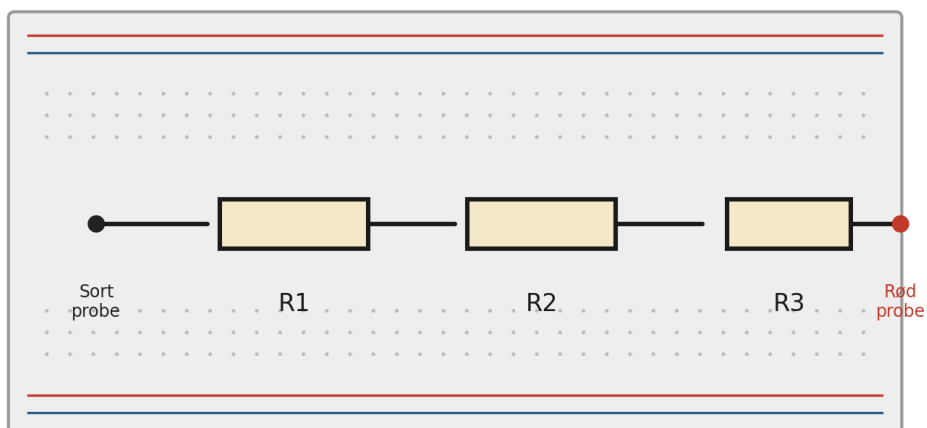


Figure 24, sådan kan det se ud fysisk på et breadboard: modstandene sidder på række.

Og i virkeligheden skal det opstilles sådan her:

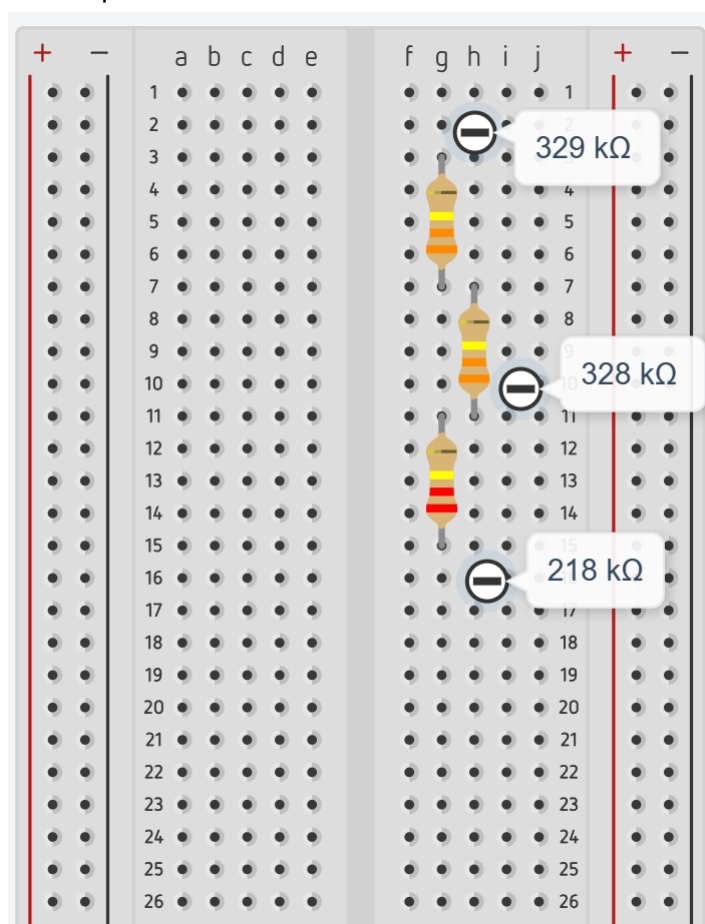


Figure 25, sådan kan det se ud fysisk på et breadboard 2

VÆR OPMÆRKSOM:

Husk at vende modstande den samme retning, som i figur 10

Formel for serieforbindelse

$$R_{\text{total}} = R_1 + R_2 + R_3 + \dots + R_n$$

Udregningseksempel

Med $R_1 = 218 \text{ k}\Omega$, $R_2 = 329 \text{ k}\Omega$ og $R_3 = 328 \text{ k}\Omega$ i serie:

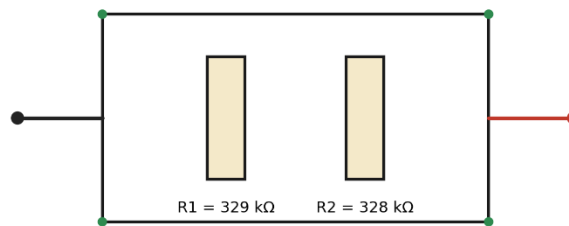
$$R_{\text{total}} = 218 + 329 + 328 = 875 \text{ k}\Omega$$

Måler du fx $881 \text{ k}\Omega$ i praksis, er det kun ca. 0,7% fra den teoretiske værdi — det er helt normalt pga. modstandstolerance og lidt måleusikkerhed.

4 Måling af parallelforbindelse

Når modstande sidder i parallel, deler de de samme to punkter (knuder) i kredsløbet — strømmen har flere veje at løbe.

Parallelforbindelse: begge modstande deler samme to punkter



De grønne punkter er de fælles knuder — mål mellem den venstre og den højre probe for R_{total}

Figure 26, to modstande i parallel. De grønne punkter er de fælles knuder, mål mellem probernes yderpunkter.

Sådan ville det se ud på et breadboard.

Begge modstandes ben sidder i de samme to rækker — det er det, der gør dem parallelle

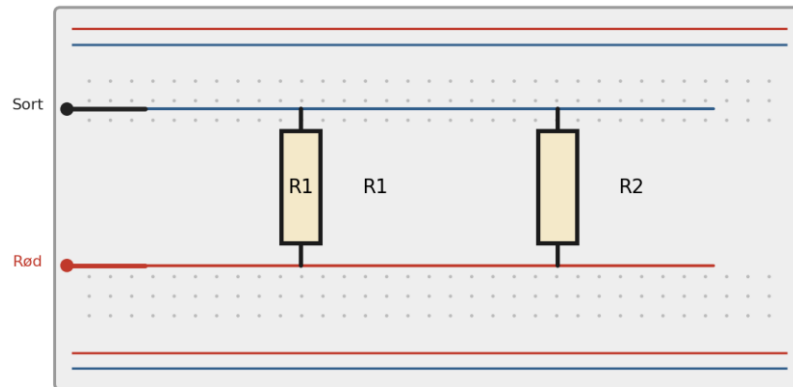


Figure 27, På breadboard: begge modstandes ben sidder i de samme to rækker.

Og i virkeligheden skal det opstilles sådan her:

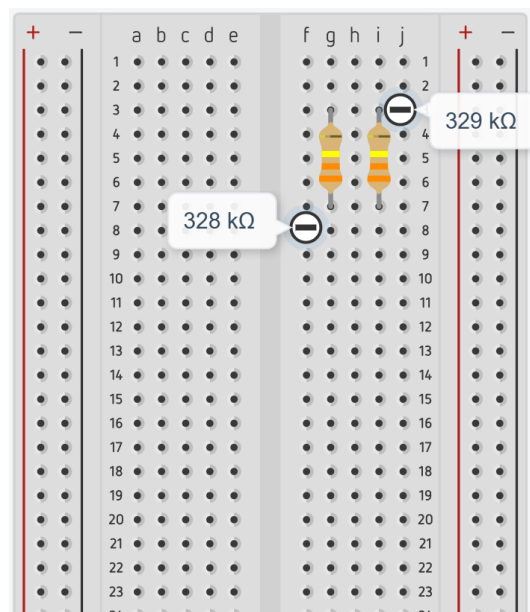


Figure 28, parallelforbindelse fysisk

VÆR OPMÆRKSOM:

Husk at vende modstande den samme retning, som i figur 13

Formel for parallelforbindelse

$$\frac{1}{R_{\text{total}}} = \frac{1}{R_1} + \frac{1}{R_2} + \dots + \frac{1}{R_n}$$

For netop to modstande kan man bruge genvejen (produkt over sum):

$$R_{\text{total}} = (R_1 \times R_2) \div (R_1 + R_2)$$

Udregningseksempel

Med $R_1 = 329 \text{ k}\Omega$ og $R_2 = 328 \text{ k}\Omega$ i parallel:

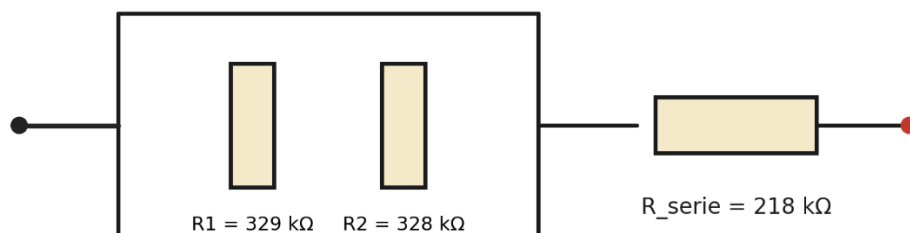
$$R_{\text{total}} = (329 \times 328) \div (329 + 328) = 107.912 \div 657 \approx 164,3 \text{ k}\Omega$$

Bemærk at resultatet ($164,3 \text{ k}\Omega$) er lavere end begge de enkelte modstande — det er altid tilfældet ved parallelforbindelse.

5 Måling af kombineret serie- og parallelforbindelse

I praksis møder man ofte kredsløb, hvor nogle modstande sidder i parallel, og resultatet af det sidder i serie med en tredje modstand (som i figur b i dit pensum).

Kombination: to modstande i parallel, derefter én modstand i serie



$$\text{Formel: } R_{\text{total}} = (R_1 \times R_2) \div (R_1 + R_2) + R_{\text{serie}}$$

Figure 29, To modstande i parallel (R_1 , R_2), efterfulgt af en modstand i serie (R_{serie}).

Det ser sådan ud på et breadboard

Breadboard-opsætning: R1 og R2 i parallel, derefter R_serie i serie

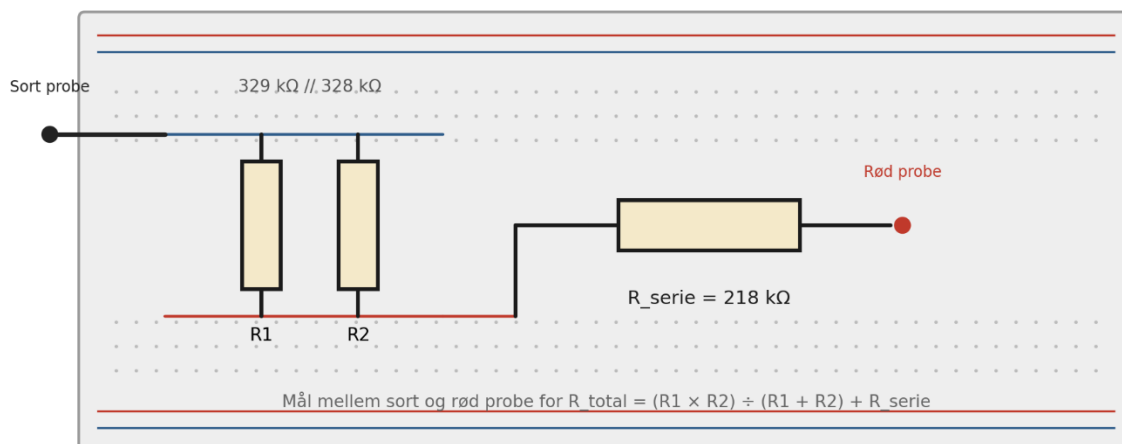


Figure 30, på breadboard, serie- og parallelforbindelse

Og i virkeligheden skal det opstilles sådan her:

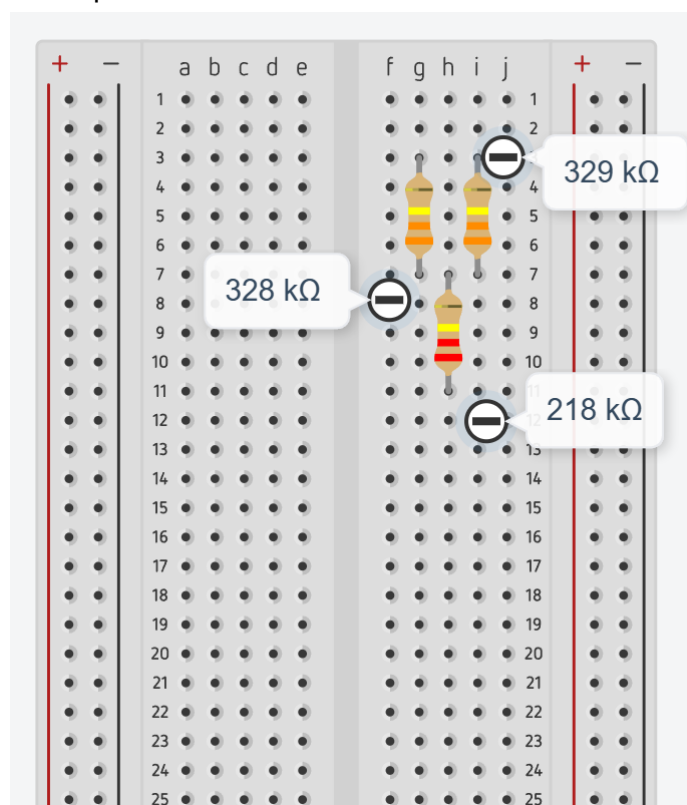


Figure 31, på breadboard, serie- og parallelforbindelse

VÆR OPMÆRKSOM:

Husk at vende modstande den samme retning, som i figur 16

Metode: regn i trin

- Trin 1: Udregn parallelblokken for sig ($R_1 // R_2$)
- Trin 2: Læg resultatet sammen med serie-modstanden

Udregningseksempel

Med $R_1 = 329 \text{ k}\Omega$, $R_2 = 328 \text{ k}\Omega$ i parallel, og $R_{\text{serie}} = 218 \text{ k}\Omega$:

$R_1 // R_2 = (329 \times 328) \div (329 + 328) \approx 164,3 \text{ k}\Omega$ $R_{\text{total}} = 164,3 + 218 = 382,3 \text{ k}\Omega$
--

En måling på fx $387 \text{ k}\Omega$ ligger kun ca. 1,2% fra denne teoretiske værdi — inden for normal tolerance.

6 Tjekliste før du måler

- Er kredsløbet helt strømløst (ingen batteri/forsyning tilsluttet)?
- Sidder den sorte ledning i COM og den røde i Ω/V ?
- Står drejeknappen på Ω ?
- Rører begge målespidser rent metal (ikke isolering) på modstandens ben?
- Har du ventet til tallet på skærmen har stabiliseret sig, før du noterer værdien?

7 Sammenligning: teoretisk vs. målt værdi

Kredsløb	Teoretisk R_{total}	Målt R_{total}
Serie ($218+329+328 \text{ k}\Omega$)	$875 \text{ k}\Omega$	$881 \text{ k}\Omega$
Parallel ($329 // 328 \text{ k}\Omega$)	$164,3 \text{ k}\Omega$	$164,6 \text{ k}\Omega$
serie- og parallelforbindelse ($329//328 + 218 \text{ k}\Omega$)	$382,3 \text{ k}\Omega$	$387 \text{ k}\Omega$

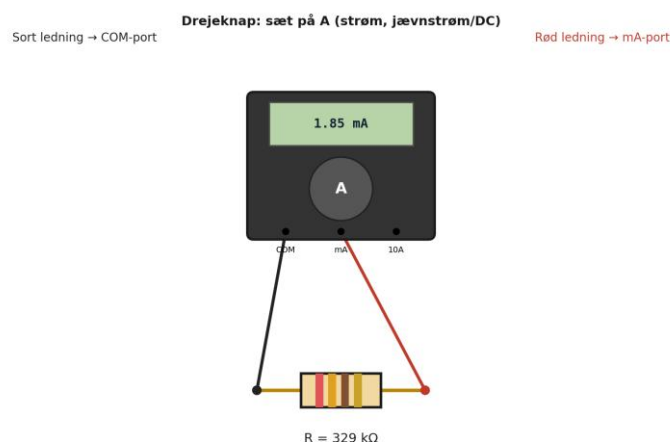
Sådan måler du modstand med multimeter med strøm

Strøm måles anderledes end modstand og spænding: multimeteret skal sidde i serie med kredsløbet, altså inde i selve ledningsvejen, så hele strømmen løber igennem multimeter. Ved modstand og spænding kobler man derimod parallelt (uden om komponenten).

Kredsløbet skal derfor brydes ét sted, og multimeteret sættes ind der som et ekstra led i kæden.

Sådan stiller du multimeteret op til strømmåling

- Sluk/afbryd forsyningen, før du ændrer på kredsløbet
- Sort ledning i COM-porten (som normalt)
- Rød ledning i mA-porten for små strømme (typisk under ca. 200 mA)
- Skift til 10A-porten, hvis du forventer en større strøm — ellers risikerer du at sprænge multimeterets sikring
- Drej vælgerknappen om på A (jævnstrøm/DC, hvis det er et DC-kredsløb)



Multimeteret skal sidde i serie med kredsløbet — IKKE parallelt som ved spænding/modstand!
Brug mA-porten til små strømme; skift til 10A-porten hvis strømmen er over ca. 200 mA

Figure 32, Opsætning til strømmåling: sort i COM, rød i mA-porten, drejeknap på A.

Sådan kobler du multimeteret ind i kredsløbet

Strømmåling: multimeteret (som ampereter) sidder i serie i kredsløbet

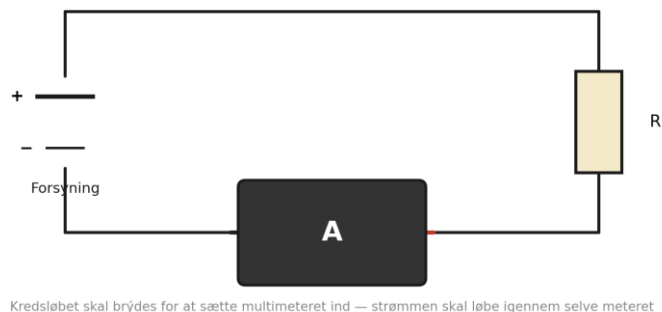


Figure 33, Multimeteret (ampereteret) sidder i serie, strømmen løber gennem det, ikke udenom.

På på et breadboard (fumlebræt).

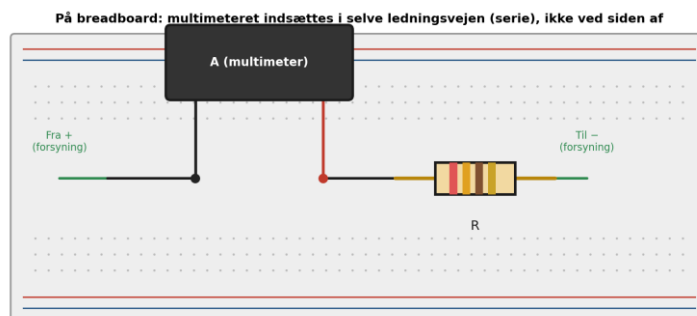


Figure 34, På breadboard: multimeteret indsættes midt i ledningsvejen, ikke ved siden af kredsløbet.

Formel: Ohms lov (strøm)

Hvor I er strømmen i ampere (A), U er spændingen i volt (V), og R er modstanden i ohm (Ω).

$$I = U \div R$$

Udregningseksempel

Med en forsyningsspænding på $U = 9\text{ V}$ og en modstand på $R = 4,7\text{ k}\Omega$:

$$I = 9 \div 4.700 = 0,001915\text{ A} = 1,915\text{ mA}$$

Multimeteret bør altså vise en værdi tæt på 1,9 mA. Afviger den målte strøm meget herfra, kan det skyldes modstandens tolerance, et dårligt kontaktpunkt, eller at forsyningsspændingen ikke præcis er 9 V (mål evt. spændingen separat for at tjekke).

Vigtige sikkerhedspunkter

- Kobl ALDRIG multimeteret parallelt over en spændingskilde, mens det står i A-tilstand, det kan skabe kortslutning og ødelægge meteret eller udløse dets sikring.
- Start altid med den højeste strømport (10A), hvis du er i tvivl om størrelsen — skift ned til mA, hvis udslaget er lille.
- Tjek at sikringen i multimeteret er intakt, hvis du pludselig ikke får noget udslag ved strømmåling.

Tjekliste før du måler

- Er kredsløbet helt strømløst (ingen batteri/forsyning tilsluttet)?
- Sidder den sorte ledning i COM og den røde i Ω/V ?
- Står drejeknappen på Ω ?
- Rører begge målespidser rent metal (ikke isolering) på modstandens ben?
- Har du ventet til tallet på skærmen har stabiliseret sig, før du noterer værdien?

Litteraturliste

- Ohm's Law Calculator – Mateusz Mucha (Omni Calculator) – <https://www.omnicalculator.com/physics/ohms-law>
- Series Resistor Calculator – Bogna Szyk & Álvaro Díez (Omni Calculator) – <https://www.omnicalculator.com/physics/series-resistor>
- Parallel Resistor Calculator – Bogna Szyk & Álvaro Díez (Omni Calculator) – <https://www.omnicalculator.com/physics/parallel-resistor>
- Big-Beautiful-Board – CN-Thrige – <https://github.com/CN-Thrige/Big-Beautiful-Board>

Undervisningsmateriale

Forfatter: Lkaa, Erhvervsakademi Aarhus, ITEK 1. semester

- Elektronik (Embedded) – noter om strøm/spænding/modstand og bølgeteori (Semesterstartsprøve_Elektronik.pdf)
- Mikrocontroller gennem tiden (Upro_gennem_tiden.pdf)
- Scope1 (scope1.pptx)
- Spændingsdeler Pico (Spændingsdeler_Pico.xlsx)
- Statemachine v2 (Statemachine_v2.pptx)
- Voltmeter analog og digitalt (Voltmeter_analog_og_digitalt.pptx)
- 7-segment (7_segment.pptx)
- AD converter Pico (AD_converter_Pico.pptx)
- GY53 Presentation (GY53_Presentation.pptx)
- Lyskryds (Lyskryds.pptx)
- Pico print (Pico_print.pptx)
- Pull up and down (Pull_up_and_DOWN.pptx)
- PWM (PWM.pptx)